Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Introducción a Sistemas Distribuidos

Taller de Evaluación de Rendimiento MPI

Integrantes:
Ana Sofía Arboleda Convers
Paula Gabriela Losada Campo
Daniel Felipe Ramírez Vargas
Guillermo Andrés Aponte Cárdenas
Samuel Pico Durán

Docente: John Jairo Corredor

Fecha: 20 de marzo de 2026

Índice

1. Objetivos **3**
1.1. Objetivo General . . . 3
1.2. Objetivos Específicos . . . 3

2. Marco Teórico **4**
2.1. Sistema de ficheros compartidos . . . 4
2.2. Computación Paralela . . . 4
2.3. MPI (Message Passing Interface) . . . 5
2.4. OpenMP . . . 5
2.5. Métricas de Rendimiento . . . 6

3. Especificaciones de los Equipos **7**

4. Procedimiento de Creación del Clúster **7**
4.1. Obtención de Direcciones IP . . . 8
4.2. Mapeo de Nombres a Direcciones IP (`/etc/hosts`) . . . 8
4.3. Configuración de Llaves SSH sin Contraseña . . . 9
4.3.1. Generación del par de llaves . . . 9
4.3.2. Distribución de la llave pública . . . 10
4.3.3. Verificación de la conexión . . . 10
4.4. Sistema de Archivos Compartido (NFS) . . . 10
4.4.1. Instalación de NFS en todas las máquinas . . . 11
4.4.2. Creación del directorio compartido (solo en el Master) . . . 11
4.4.3. Configuración de exportaciones NFS (solo en el Master) . . . 11
4.4.4. Inicio del servicio NFS (solo en el Master) . . . 12
4.4.5. Montaje del directorio compartido en los Workers . . . 13
4.5. Instalación de OpenMPI . . . 13
4.6. Configuración del Hostfile de MPI . . . 14
4.7. Compilación de los Archivos Fuente . . . 14
4.8. Ejecución del Programa . . . 15

5. Diseño de Experimentos **16**
5.1. Variables del Experimento . . . 17
5.2. Constantes del Experimento . . . 18
5.3. Criterio de Divisibilidad . . . 19
5.4. Archivos Fuente del Experimento . . . 20
5.4.1. moduloMPI.h Declaraciones compartidas . . . 20
5.4.2. moduloMPI.c Funciones compartidas de soporte . . . 20
5.4.3. mxmOmpMPIfxc.c Algoritmo Filas × Columnas . . . 21
5.4.4. mxmOmpMPIfxt.c Algoritmo Filas × Transpuesta . . . 21
5.4.5. Makefile Compilación consistente . . . 22
5.4.6. lanzadorMPI.pl Automatización del experimento . . . 22

6. Análisis de Resultados **22**
6.1. Tiempo Promedio de Ejecución 23
6.2. Speedup 24
6.3. Eficiencia 27
6.4. Overhead de Comunicación 31
6.5. Comparación entre Algoritmos FxC y FxT 35
6.6. Limitaciones del Experimento 37
6.7. Discusión General 38

7. Conclusiones y Recomendaciones **39**

Resumen

Este trabajo presenta la evaluación de rendimiento de algoritmos de multiplicación de matrices en entornos de computación paralela, comparando implementaciones secuenciales y paralelas mediante el uso de MPI y OpenMP. El estudio se desarrolló sobre un clúster de cómputo con arquitectura homogénea y sistema operativo Linux, permitiendo analizar el impacto de diferentes configuraciones de paralelismo, como el número de procesos y hilos, así como el tamaño de las matrices.

Se implementaron dos variantes algorítmicas: el método clásico de filas por columnas y una versión optimizada basada en la transposición de matrices para mejorar la localidad de memoria. A través de un diseño experimental controlado, se ejecutaron múltiples pruebas repetidas, recolectando métricas como tiempo de ejecución, speedup, eficiencia y overhead de comunicación.

Los resultados evidencian que el paralelismo mejora el desempeño, especialmente para tamaños de problema grandes, aunque introduce costos asociados a la coordinación entre procesos. Se observa además que el uso de hilos (OpenMP) ofrece mayor estabilidad y eficiencia en comparación con procesos distribuidos (MPI), mientras que la variante con transposición logra un mejor rendimiento absoluto debido a un uso más eficiente de la caché. Finalmente, el estudio destaca la importancia de balancear el modelo de paralelismo, la granularidad del problema y el patrón de acceso a memoria para obtener un desempeño óptimo en sistemas de alto rendimiento.

1. Objetivos

1.1. Objetivo General

Evaluar y comparar el rendimiento de algoritmos de multiplicación de matrices en versiones secuencial y paralela, mediante el uso de tecnologías como MPI y OpenMP, sobre diferentes sistemas de cómputo con sistema operativo Linux, con el fin de analizar su desempeño y obtener conclusiones fundamentadas.

1.2. Objetivos Específicos

- Comparar dos algoritmos de multiplicación de matrices en términos de rendimiento.
- Implementar y evaluar versiones secuenciales y paralelas de los algoritmos utilizando C, MPI y OpenMP.
- Analizar el comportamiento de los algoritmos en distintos sistemas de cómputo con sistema operativo Linux.
- Diseñar y ejecutar una batería de experimentos que permita medir el desempeño bajo diferentes configuraciones (tamaño de matrices, número de hilos, etc.).
- Interpretar los resultados obtenidos mediante métricas de desempeño adecuadas.

- Elaborar un informe técnico que incluya el análisis, conclusiones y recomendaciones basadas en los resultados experimentales.

2. Marco Teórico

2.1. Sistema de ficheros compartidos

Un sistema de ficheros compartidos es una infraestructura que permite a múltiples usuarios o procesos acceder, almacenar y manipular archivos de manera concurrente a través de una red. Este tipo de sistema es fundamental en entornos distribuidos, donde los recursos no se encuentran centralizados, sino que están repartidos entre diferentes nodos interconectados [1].

En un sistema distribuido, los sistemas de ficheros compartidos proporcionan transparencia de acceso, lo que significa que los usuarios pueden interactuar con los archivos como si estuvieran almacenados localmente, independientemente de su ubicación física. Esta característica mejora la usabilidad y facilita la colaboración entre múltiples usuarios [2].

Existen diferentes arquitecturas para implementar sistemas de ficheros compartidos, entre las cuales destacan los sistemas cliente-servidor y los sistemas completamente distribuidos. En el modelo cliente-servidor, un servidor central administra los archivos y los clientes realizan solicitudes de acceso. Por otro lado, en los sistemas distribuidos puros, todos los nodos pueden actuar tanto como clientes como servidores, lo que mejora la escalabilidad y tolerancia a fallos [1].

Uno de los principales desafíos en los sistemas de ficheros compartidos es garantizar la consistencia de los datos cuando múltiples usuarios acceden simultáneamente a los mismos archivos. Para abordar este problema, se emplean mecanismos de control de concurrencia, como bloqueos (locks) y protocolos de coherencia de caché [3].

Además, estos sistemas deben considerar aspectos de seguridad, como la autenticación de usuarios, el control de acceso y la protección de datos. También es importante la eficiencia en el acceso a los archivos, lo cual se logra mediante técnicas como el almacenamiento en caché, la replicación de datos y la distribución de cargas [1].

Finalmente, los sistemas de ficheros compartidos son ampliamente utilizados en aplicaciones modernas como la computación en la nube, sistemas colaborativos y entornos de alto rendimiento, donde múltiples usuarios requieren acceso simultáneo a grandes volúmenes de información.

2.2. Computación Paralela

La computación paralela es un paradigma de procesamiento en el cual múltiples operaciones o tareas se ejecutan simultáneamente utilizando varios recursos computacionales, con el objetivo de reducir el tiempo de ejecución y mejorar el rendimiento de los sistemas. Este enfoque se basa en la descomposición de un problema en subproblemas más pequeños que pueden ser resueltos de manera concurrente [7].

A diferencia de la computación secuencial, donde las instrucciones se ejecutan una tras otra, la computación paralela permite la ejecución simultánea de instrucciones en diferentes unidades de procesamiento, como múltiples núcleos de un procesador, procesadores independientes o incluso sistemas distribuidos [4]. Este tipo de computación es especialmente relevante en aplicaciones que requieren un alto poder de cómputo, como

simulaciones científicas, procesamiento de grandes volúmenes de datos y algoritmos de inteligencia artificial.

Existen diferentes modelos de computación paralela, entre los que se destacan el paralelismo de datos, donde la misma operación se aplica a múltiples datos de forma simultánea, y el paralelismo de tareas, donde diferentes tareas se ejecutan en paralelo. Asimismo, los sistemas paralelos pueden clasificarse según su arquitectura, como memoria compartida o memoria distribuida, siendo esta última comúnmente implementada mediante estándares como MPI (Message Passing Interface) [5].

En este contexto, la computación paralela constituye un componente fundamental en el desarrollo de sistemas de alto rendimiento (HPC), permitiendo aprovechar eficientemente los recursos computacionales disponibles para resolver problemas complejos en menor tiempo.

2.3. MPI (Message Passing Interface)

MPI (Message Passing Interface) es un estándar ampliamente utilizado para la programación en sistemas de memoria distribuida, que permite la comunicación y coordinación entre múltiples procesos que se ejecutan de manera paralela. Este estándar define un conjunto de funciones y protocolos para el intercambio de mensajes entre procesos, facilitando el desarrollo de aplicaciones paralelas escalables y eficientes [5].

A diferencia de los modelos de memoria compartida, en MPI cada proceso posee su propio espacio de memoria local, por lo que la comunicación se realiza explícitamente mediante el envío y recepción de mensajes. Este enfoque permite un mayor control sobre la sincronización y el flujo de datos, siendo especialmente adecuado para sistemas distribuidos y clústeres de alto rendimiento (HPC) [7].

MPI soporta diferentes tipos de comunicación, incluyendo comunicación punto a punto (entre dos procesos) y comunicación colectiva (entre múltiples procesos), lo que permite implementar una amplia variedad de algoritmos paralelos. Además, proporciona mecanismos para la sincronización de procesos, manejo de grupos y topologías de comunicación [4].

Debido a su portabilidad, eficiencia y amplio soporte en la comunidad científica e industrial, MPI se ha consolidado como uno de los estándares más importantes en la computación paralela, siendo implementado en diversas bibliotecas como Open MPI y MPICH, y utilizado en aplicaciones que requieren alto rendimiento computacional.

2.4. OpenMP

OpenMP (Open Multi-Processing) es una interfaz de programación de aplicaciones (API) diseñada para el desarrollo de programas paralelos en sistemas de memoria compartida. Este modelo permite a los desarrolladores implementar paralelismo mediante el uso de directivas, bibliotecas y variables de entorno que facilitan la creación y gestión de múltiples hilos de ejecución dentro de un mismo programa [4, 8].

A diferencia de modelos como MPI, donde los procesos se comunican mediante el envío explícito de mensajes, OpenMP se basa en el uso de memoria compartida, lo que permite que los hilos accedan directamente a los mismos datos. Esto simplifica la programación paralela, ya que reduce la necesidad de gestionar explícitamente la comunicación entre procesos, aunque introduce desafíos relacionados con la sincronización y la concurrencia [7].

El modelo de programación de OpenMP se fundamenta en la paralelización incremental, donde un programa secuencial puede transformarse en paralelo mediante la inserción de directivas (como `#pragma omp`) en regiones específicas del código. Además, OpenMP proporciona mecanismos para la sincronización, como barreras, secciones críticas y variables privadas, que permiten controlar el acceso a los recursos compartidos [8].

Debido a su facilidad de uso, portabilidad y eficiencia en arquitecturas multicore, OpenMP se ha convertido en una herramienta ampliamente utilizada en aplicaciones científicas, ingeniería y procesamiento de datos, especialmente en entornos donde la memoria compartida es predominante.

2.5. Métricas de Rendimiento

Las métricas de rendimiento son indicadores cuantitativos utilizados para evaluar el comportamiento y la eficiencia de algoritmos y sistemas computacionales, particularmente en entornos de computación paralela. Estas métricas permiten analizar el impacto de factores como la paralelización, el uso de recursos y las estrategias de implementación sobre el tiempo de ejecución [7].

En el contexto de este trabajo, la métrica principal utilizada es el *tiempo de ejecución*, medido en microsegundos mediante funciones de temporización. Esta métrica permite comparar directamente el desempeño de diferentes algoritmos de multiplicación de matrices (filas por columnas y filas por transpuesta), así como evaluar el efecto del número de procesos MPI y del número de hilos OpenMP en el rendimiento total del sistema.

Adicionalmente, el diseño experimental considera la repetición de cada prueba múltiples veces con el fin de obtener resultados más confiables y reducir la variabilidad inherente a la ejecución en sistemas paralelos. Esto permite analizar tendencias de rendimiento y realizar comparaciones consistentes entre distintas configuraciones experimentales.

Otras métricas fundamentales en computación paralela incluyen el *speedup* y la *eficiencia*. El speedup mide la aceleración obtenida al ejecutar un programa en paralelo en comparación con su versión secuencial, mientras que la eficiencia evalúa el grado de aprovechamiento de los recursos computacionales disponibles [4]. Estas métricas serán utilizadas en la sección de análisis de resultados para complementar la evaluación del desempeño basada en el tiempo de ejecución.

Asimismo, se considera el *overhead de comunicación*, el cual representa el costo adicional asociado a la coordinación y transferencia de datos entre procesos en un entorno distribuido. Este factor es especialmente relevante en implementaciones que utilizan MPI, donde la comunicación entre nodos puede impactar significativamente el rendimiento global del sistema.

Como complemento a las métricas anteriores, para expresar el rendimiento absoluto de cada configuración también se utilizó la métrica de *GFLOPS* (*giga floating-point operations per second*). En la multiplicación de dos matrices cuadradas de tamaño $N \times N$, el costo aritmético se aproximó como $2N^3$ operaciones de punto flotante: una multiplicación y una suma por cada iteración del producto interno. A partir de esta convención, el rendimiento se calculó mediante:

$$\text{GFLOPS} = \frac{2N^3}{T \cdot 10^9}$$

donde T es el tiempo de ejecución expresado en segundos. Esta aproximación se justifica porque cada elemento de la matriz resultado C se obtiene como el producto punto entre una fila de A y una columna de B, lo que implica aproximadamente N multiplicaciones y N sumas por elemento. Como la matriz resultado contiene N^2 elementos, el número total de operaciones se aproxima por $N^2(2N) = 2N^3$. Como las mediciones base del experimento se registraron en microsegundos, para este cálculo se realizó primero la conversión $T = t_{\mu s}/10^6$. De este modo, la métrica refleja cuántos miles de millones de operaciones de punto flotante por segundo logró sostener cada implementación y permite comparar de forma directa el rendimiento bruto entre variantes algorítmicas y configuraciones de paralelismo.

El uso conjunto de estas métricas permite realizar una evaluación integral del comportamiento de los algoritmos paralelos, identificar posibles cuellos de botella y fundamentar las conclusiones del experimento en términos de eficiencia y escalabilidad.

3. Especificaciones de los Equipos

El clúster empleado para la experimentación estuvo conformado por un total de 5 equipos de cómputo, todos ellos pertenecientes a la sala Alan Turing de la universidad, ubicada en el edificio José Gabriel Maldonado, en la zona de laboratorios. Los equipos fueron interconectados físicamente mediante una misma red LAN cableada, lo que proporcionó un entorno de comunicación local estable y consistente para la ejecución del experimento distribuido.

Los 5 equipos presentaban especificaciones de hardware homogéneas, es decir, todos los nodos disponían de la misma capacidad de cómputo. En particular, cada computador contaba con 8 cores de procesamiento. Esta uniformidad reduce el sesgo asociado a diferencias entre máquinas y permite atribuir las variaciones observadas en el tiempo de ejecución a la configuración de paralelismo evaluada, y no a heterogeneidad del entorno de cómputo.

La toma de datos se realizó además en condiciones controladas, procurando igualdad de operación para todos los nodos participantes. Durante la recolección de las mediciones no hubo contaminación de carga externa sobre los equipos, es decir, no se ejecutaron tareas adicionales que interfirieran de manera significativa con el experimento. De este modo, los tiempos registrados reflejan de forma más fiel el comportamiento de los programas bajo estudio dentro del clúster configurado.

4. Procedimiento de Creación del Clúster

El clúster se construyó sobre los equipos de la sala, conectados por red local, todos con Ubuntu Linux como sistema operativo. El proceso de configuración sigue el siguiente orden: primero se obtienen las direcciones IP de cada máquina y se registran en el archivo de hosts, luego se habilita la comunicación segura sin contraseña entre nodos mediante SSH, después se configura el sistema de archivos compartido con NFS, y finalmente se instala y configura OpenMPI. Cada uno de estos pasos se describe a continuación.

4.1. Obtención de Direcciones IP

Para que los nodos puedan comunicarse entre sí, es necesario conocer la dirección IP de cada máquina dentro de la red local. Se utilizó el siguiente comando en cada equipo:

```
ifconfig
```

Donde:

- `ifconfig` — (*interface configuration*) para consultar y configurar interfaces de red. Muestra información de cada interfaz activa: dirección IP, máscara de red, dirección MAC y estadísticas de tráfico.

La dirección relevante es la que aparece como `inet` en la interfaz de red local. Se registró la IP de cada equipo para el paso siguiente. Las IPs obtenidas fueron:

```
10.195.23.70   master
10.195.23.160  nodo0
10.195.23.161  nodo1
10.195.23.39   nodo2
```

4.2. Mapeo de Nombres a Direcciones IP (`/etc/hosts`)

Se utilizó el archivo `/etc/hosts` como una tabla local de resolución de nombres: antes de consultar un servidor, el sistema busca aquí si el nombre solicitado tiene una IP asignada. Agregar todos los nodos del clúster en este archivo permite que cada máquina reconozca a las demás por su nombre en lugar de por IP.

> **Importante:** Este proceso debe ejecutarse en *todas* las máquinas del clúster, incluyendo el Master y nodos.

Se edita el archivo con el siguiente comando:

```
sudo nano /etc/hosts
```

Donde:

- `nano` — editor de texto en terminal, sencillo en Ubuntu.
- `/etc/hosts` — archivo del sistema que mapea nombres a IPs; solo puede modificarlo el administrador (`sudo`).

Dentro del archivo, al final del contenido existente, se agregan las líneas con la IP y el nombre asignado a cada nodo. Los nombres elegidos fueron `master` para el nodo coordinador y `nodo0`, `nodo1`, `nodo2` para los Workers:

```
10.195.23.70   master
10.195.23.160  nodo0
10.195.23.161  nodo1
10.195.23.39   nodo2
```

Para guardar en `nano`: `Ctrl+O`, luego `Enter` para confirmar, y `Ctrl+X` para salir.

Para verificar que la resolución de nombres funciona correctamente, se utiliza el comando `ping` hacia cada nodo:

```
ping nodo0 -c 4
```

Donde:

- `ping` — envía paquetes ICMP de prueba a una dirección o nombre de host para verificar conectividad.
- `nodo0` — nombre del nodo destino (resuelto gracias a `/etc/hosts`).
- `-c 4` — limita el número de paquetes enviados a 4; sin esta opción, `ping` se ejecuta indefinidamente.

Una salida exitosa muestra `0% packet loss`, lo que confirma que la comunicación entre nodos funciona y que el mapeo de nombres es correcto.

4.3. Configuración de Llaves SSH sin Contraseña

MPI utiliza SSH (*Secure Shell*) para lanzar procesos de forma remota en los nodos Workers desde el Master. SSH es un protocolo de comunicación cifrada que permite controlar otra máquina a través de la red de forma segura. Para que MPI funcione de manera automática y sin interrupciones, SSH no debe solicitar contraseña en ninguna conexión entre nodos.

La solución es usar *autenticación por llave pública*: cada máquina genera un par de llaves criptográficas (una privada que nunca sale del equipo y una pública que se distribuye a los demás nodos). Cuando un nodo intenta conectarse a otro, ambos verifican sus llaves y, si coinciden, la conexión se establece sin necesidad de contraseña.

Importante: Este proceso debe realizarse en *todas* las máquinas. Además, cada nodo debe poder conectarse a sí mismo y a todos los demás sin contraseña, incluyendo el Master conectándose a sí mismo (`ssh master`).

4.3.1. Generación del par de llaves

En **cada máquina** se genera el par de llaves con:

```
ssh-keygen -t rsa -b 2048
```

Donde:

- `ssh-keygen` — herramienta para generar, gestionar y convertir llaves de autenticación SSH.
- `-t rsa` — especifica el tipo de algoritmo criptográfico; RSA es el estándar más compatible.
- `-b 2048` — define el tamaño de la llave en bits; 2048 bits ofrece un balance adecuado entre seguridad y rendimiento.

El comando solicita la ruta donde guardar las llaves (presionar `Enter` para aceptar la ruta por defecto `~/.ssh/id_rsa`) y una contraseña de protección (presionar `Enter` dos veces para dejarlo vacío, lo cual es necesario para conexiones automáticas sin intervención manual).

Esto genera dos archivos en `~/.ssh/`:

- `id_rsa` — llave **privada**. Nunca debe compartirse ni salir del equipo.
- `id_rsa.pub` — llave **pública**. Es la que se copia a los demás nodos.

4.3.2. Distribución de la llave pública

Desde **cada máquina**, se copia la llave pública a todos los demás nodos (incluyendo a sí mismo):

```
ssh-copy-id master
ssh-copy-id nodo0
ssh-copy-id nodo1
ssh-copy-id nodo2
```

Donde:

- `ssh-copy-id` — copia la llave pública del equipo local al archivo `~/.ssh/authorized_keys` del nodo destino. SSH verifica este archivo para permitir conexiones sin contraseña.
- `master, nodo0...` — nombre del nodo destino, resuelto gracias al archivo `/etc/hosts` configurado anteriormente.

Este comando pedirá la contraseña del usuario del nodo destino una única vez (la última vez que se pedirá).

4.3.3. Verificación de la conexión

Para confirmar que la autenticación sin contraseña funciona correctamente, se intenta conectar a cada nodo:

```
ssh nodo0
```

Si la conexión se establece directamente sin solicitar contraseña, el proceso fue exitoso. Para salir de la sesión remota:

```
exit
```

4.4. Sistema de Archivos Compartido (NFS)

NFS permite que todos los nodos del clúster accedan al mismo directorio como si fuera local en cada máquina. Esto es esencial para que los ejecutables compilados y los resultados de los experimentos estén disponibles en todos los nodos sin necesidad de copiarlos manualmente. La arquitectura es servidor-cliente: el Master exporta el directorio y los Workers lo montan.

4.4.1. Instalación de NFS en todas las máquinas

En **todas las máquinas** (Master y Workers):

```
sudo apt install -y nfs-common
```

Donde:

- `apt install` — gestor de paquetes de Ubuntu; descarga e instala software desde los repositorios oficiales.
- `-y` — responde "sí" automáticamente a todas las confirmaciones durante la instalación.
- `nfs-common` — paquete con las herramientas necesarias para montar sistemas de archivos NFS remotos.

Adicionalmente, en el **nodo Master** se instala el servidor NFS:

```
sudo apt install -y nfs-kernel-server
```

- `nfs-kernel-server` — implementación del servidor NFS en el kernel de Linux; permite exportar directorios a otros equipos.

4.4.2. Creación del directorio compartido (solo en el Master)

Se crea el directorio `/Almacen` que será exportado a todos los nodos:

```
sudo mkdir /Almacen
sudo chown -R sistemas:sistemas /Almacen
```

Donde:

- `mkdir` — crea un nuevo directorio.
- `/Almacen` — ruta del directorio en la raíz del sistema de archivos; debe existir con la misma ruta en todos los nodos para garantizar coherencia de rutas al ejecutar MPI.
- `chown` — cambia el propietario de un archivo o directorio (*change owner*).
- `-R` — aplica el cambio de forma recursiva a todos los archivos y subdirectorios dentro.
- `sistemas:sistemas` — nombre del usuario y grupo del sistema; se asigna como propietario para poder leer y escribir en el directorio sin necesidad de `sudo`.

4.4.3. Configuración de exportaciones NFS (solo en el Master)

Se edita el archivo `/etc/exports`, que define qué directorios exporta el servidor NFS y a quién tiene acceso:

```
sudo nano /etc/exports
```

Se agrega una línea por cada nodo Worker con la IP real del equipo:

```
/Almacen 10.195.23.160(rw,no_root_squash,subtree_check,fsid=0)
/Almacen 10.195.23.161(rw,no_root_squash,subtree_check,fsid=0)
/Almacen 10.195.23.39(rw,no_root_squash,subtree_check,fsid=0)
```

El significado de cada opción entre paréntesis es:

- `rw` — (*read-write*) el cliente puede leer y escribir en el directorio exportado.
- `no_root_squash` — permite que el usuario `root` del cliente opere con privilegios de root en el directorio exportado. Sin esta opción, root del cliente se mapea a un usuario sin privilegios, lo que puede causar errores de permiso al compilar o ejecutar.
- `subtree_check` — el servidor verifica que el archivo solicitado pertenece efectivamente al directorio exportado, añadiendo una capa de seguridad.
- `fsid=0` — asigna un identificador único al sistema de archivos exportado, necesario para que NFS lo identifique correctamente.

Para aplicar los cambios y exportar el directorio sin reiniciar el servicio:

```
sudo exportfs -rv
```

Donde:

- `exportfs` — herramienta para gestionar las exportaciones NFS activas.
- `-r` — re-exporta todos los directorios listados en `/etc/exports`.
- `-v` — modo verbose; muestra en consola qué directorios se están exportando y a qué clientes.

4.4.4. Inicio del servicio NFS (solo en el Master)

Se habilita e inicia el servidor NFS:

```
sudo systemctl enable nfs-kernel-server
sudo systemctl start  nfs-kernel-server
sudo systemctl status nfs-kernel-server
```

Donde:

- `systemctl` — herramienta para gestionar servicios del sistema en Linux (iniciar, detener, habilitar, verificar estado).
- `enable` — configura el servicio para que inicie automáticamente al arrancar el sistema.
- `start` — inicia el servicio de forma inmediata.
- `status` — muestra el estado actual del servicio. Una salida exitosa muestra `active (running)`.

4.4.5. Montaje del directorio compartido en los Workers

En **cada nodo Worker**, se crea el punto de montaje local y se monta el directorio exportado por el Master:

```
sudo mkdir /Almacen
sudo mount master:/Almacen /Almacen
```

Donde:

- `mount` — comando que conecta (monta) un sistema de archivos remoto en un punto del árbol de directorios local.
- `master:/Almacen` — especifica el servidor NFS (`master`) y el directorio que exporta.
- `/Almacen` — punto de montaje local en el Worker; a partir de este momento, todo lo que el Master escriba en `/Almacen` será visible inmediatamente en esta ruta en todos los Workers.

4.5. Instalación de OpenMPI

OpenMPI es la implementación de código abierto del estándar MPI utilizada en este laboratorio. Proporciona las herramientas necesarias para compilar y ejecutar programas paralelos distribuidos en C.

En **todas las máquinas** del clúster se ejecuta:

```
sudo apt install -y openmpi-bin openmpi-common libopenmpi-dev
   libgtk2.0-dev
```

Los paquetes instalados son:

- `openmpi-bin` — contiene los ejecutables principales: `mpirun` (lanzador de procesos MPI), `mpicc` (compilador MPI para C) y `mpiexec`.
- `openmpi-common` — archivos de configuración compartidos y datos comunes necesarios para el funcionamiento de OpenMPI.
- `libopenmpi-dev` — librerías y archivos de cabecera (*headers*) necesarios para compilar código C que use MPI, incluyendo `mpi.h`.
- `libgtk2.0-dev` — librería de interfaces gráficas; requerida por algunas herramientas auxiliares de OpenMPI.

Para verificar que la instalación fue exitosa:

```
mpirun --version
```

La salida debe mostrar la versión instalada de OpenMPI, por ejemplo: `mpirun (Open MPI) 4.x.x`.

4.6. Configuración del Hostfile de MPI

El *hostfile* es un archivo de texto que le indica a MPI cuáles son los nodos disponibles en el clúster y cuántos procesos puede ejecutar cada uno simultáneamente (*slots*). MPI lo consulta al ejecutar `mpirun` para saber cómo distribuir los procesos.

El ZIP del profesor incluye un ejemplo llamado `filehost`. Se ubica en la carpeta compartida y se edita según la configuración del clúster:

```
nano /Almacen/filehost
```

```
master  slots=4
nodo0   slots=4
nodo1   slots=4
nodo2   slots=4
```

Donde:

- Cada línea representa un nodo del clúster identificado por su nombre (resuelto vía `/etc/hosts`).
- `slots=4` — indica cuántos procesos MPI puede alojar ese nodo de forma simultánea. Este valor debe corresponderse con el número de núcleos o hilos lógicos del procesador de cada máquina. Con 5 nodos de 4 slots cada uno, el máximo de procesos totales es $np = 5 \times 4 = 20$.

Nota sobre el proceso MASTER: En los algoritmos del ZIP, el proceso con `rank = 0` es el MASTER y **no realiza cálculos**; solo distribuye y recolecta datos. Por lo tanto, el número efectivo de Workers es $np-1$. Esto implica que la dimensión de la matriz N debe ser divisible por $(np-1)$, no por np.

4.7. Compilación de los Archivos Fuente

Los archivos fuente del experimento se ubican en la carpeta compartida para que todos los nodos accedan al mismo ejecutable:

```
cp -r /ruta/evalMxM_MPI/* /Almacen/
cd /Almacen
```

Donde:

- `cp -r` — copia archivos y directorios de forma recursiva.
- `cd` — cambia el directorio de trabajo actual (*change directory*).

La compilación se realiza desde el **nodo Master** con el `Makefile` incluido:

```
make
```

`make` lee el archivo `Makefile` y ejecuta los comandos de compilación definidos en él. En este caso compila los dos algoritmos:

```
mpicc -o mxmOmpMPIfxc mxmOmpMPIfxc.c moduloMPI.c -lm -fopenmp
mpicc -o mxmOmpMPIfxt mxmOmpMPIfxt.c moduloMPI.c -lm -fopenmp
```

El significado de cada parte del comando de compilación es:

- `mpicc` — compilador de MPI para C; es una capa sobre `gcc` que añade automáticamente las librerías y rutas necesarias para MPI.
- `-o mxmOmpMPIfxc` — nombre del ejecutable de salida.
- `mxmOmpMPIfxc.c moduloMPI.c` — archivos fuente a compilar; `moduloMPI.c` contiene las funciones compartidas por ambos algoritmos (transposición, inicialización de matrices, medición de tiempos, etc.).
- `-lm` — enlaza la librería matemática de C (`math.h`), necesaria para funciones como `sqrt`.
- `-fopenmp` — habilita el soporte de OpenMP, necesario para las directivas `#pragma omp` usadas en el cálculo paralelo dentro de cada Worker.

Para verificar que los ejecutables fueron generados:

```
ls -lh /Almacen/mxmOmpMPIfxc /Almacen/mxmOmpMPIfxt
```

4.8. Ejecución del Programa

La sintaxis general para ejecutar cualquiera de los dos algoritmos es:

```
mpirun -hostfile /Almacen/filehost -np <np> ./<ejecutable> <N>
   <nH>
```

Donde:

- `mpirun` — lanzador de procesos MPI; conecta a los nodos via SSH, inicia el número de procesos indicado y coordina la comunicación entre ellos.
- `-hostfile /compartido/filehost` — indica el archivo que lista los nodos disponibles y sus slots.
- `-np <np>` — número total de procesos MPI a lanzar, incluyendo el MASTER. Con `-np 4` se tendrán 1 MASTER y 3 Workers.
- `<ejecutable>` — `mxmOmpMPIfxc` (algoritmo Filas×Columnas) o `mxmOmpMPIfxt` (algoritmo Filas×Transpuesta).
- `<N>` — dimensión de la matriz cuadrada $N \times N$. Debe ser divisible por $(np - 1)$.
- `<nH>` — número de hilos OpenMP que usará cada Worker internamente para el cálculo. Típicamente igual al número de núcleos por máquina.

Ejemplo concreto con una matriz de 800×800, 4 procesos MPI y 2 hilos OpenMP por Worker:

```
mpirun -hostfile /Almacen/filehost -np 4 ./mxmOmpMPIfxc 800 2
```

La salida imprime el tiempo de ejecución en microsegundos. Para verificar el funcionamiento con una matriz pequeña (el programa imprime las matrices si $N < 13$):

```
mpirun -hostfile /Almacen/filehost -np 4 ./mxmOmpMPIfxc 8 1
```

5. Diseño de Experimentos

Con el fin de evaluar de manera rigurosa el rendimiento de la multiplicación de matrices en un entorno distribuido MPI con paralelismo OpenMP, se diseñó un experimento controlado que varía de forma sistemática los factores que inciden en el tiempo de ejecución. El eje central del diseño experimental es el script de automatización `lanzadorMPI.pl`, escrito en Perl, el cual define las combinaciones de parámetros a ejecutar, orquesta las invocaciones de `mpirun` y almacena los tiempos resultantes en archivos `.dat` dentro del directorio `datosDAT/`.

El script contempla dos programas bajo evaluación:

- `mxmOmpMPIfxc:` implementa el algoritmo clásico de multiplicación filas $\times$ columnas.
- `mxmOmpMPIfxt:` implementa el algoritmo de multiplicación filas $\times$ transpuesta, en el cual la matriz B se transpone previamente para favorecer la localidad espacial de caché durante el cómputo.

Ambos ejecutables se compilan mediante el `Makefile` del proyecto con el compilador `mpicc`, enlazando el módulo compartido `moduloMPI.c` y habilitando OpenMP con la bandera `-fopenmp`. Esto garantiza que las diferencias de rendimiento observadas se deban exclusivamente a la variante algorítmica y a los parámetros experimentales, mas no a diferencias de compilación.

El script `lanzadorMPI.pl` organiza la experimentación en tres escenarios diferenciados, cada uno dirigido a aislar el efecto de una variable específica:

- **Caso 1 Variación de procesos MPI entre nodos.** Se ejecuta cada programa variando el número total de procesos MPI ($np \in \{5, 17, 33\}$), distribuyéndolos entre los nodos del clúster mediante el archivo `procesosHostfile` y la política `--map-by node`. El número de hilos OpenMP se fija en $nH = 1$, de modo que el paralelismo proviene únicamente de la distribución MPI. El comando resultante es de la forma:

```
mpirun -hostfile procesosHostfile -np <NP>--map-by node
./<programa><N>1
```

- **Caso 2 Variación de hilos OpenMP por nodo.** Se fija un proceso MPI por nodo ($np = 5$, uno por cada nodo más el proceso maestro) y se varía el número de hilos OpenMP ($nH \in \{1, 4, 8\}$), usando el archivo `hilosHostFile` con un slot por nodo y la política `--map-by node`. De esta manera, el paralelismo intra-nodo queda determinado por OpenMP. El comando resultante es:

```
mpirun -hostfile hilosHostFile -np 5 --map-by node
./<programa><N><nH>
```

- **Caso 3 — Referencia base en el nodo maestro.** Se ejecuta todo el cómputo concentrado en el nodo `master`, con $np = 2$ y $nH = 1$. Se emplea $np = 2$ porque la arquitectura maestro-trabajador del código requiere al menos un proceso trabajador además del maestro (el proceso con rango 0 no participa en el cómputo de la multiplicación; únicamente inicializa las matrices, distribuye las tajadas y recopila los resultados). Este escenario constituye la línea base sobre la cual se podrán calcular posteriormente métricas comparativas. El comando es:

```
mpirun --host master:2 -np 2 ./<programa><N>1
```

Para cada combinación de algoritmo, tamaño de matriz y configuración de paralelismo, el script ejecuta 30 repeticiones independientes, siguiendo el principio de la Ley de los Grandes Números para obtener promedios estables y reducir el efecto de la variabilidad inherente al sistema operativo y a la red del clúster. Los tiempos de cada repetición se registran mediante la función `endTime()` del módulo compartido, que calcula el intervalo transcurrido con `gettimeofday()` e imprime el resultado en microsegundos por la salida estándar. El script redirige dicha salida al archivo `.dat` correspondiente.

Los archivos de datos generados siguen una convención de nombres que codifica toda la información del caso experimental. Por ejemplo, `Procesos-Pr-clasica-N-800-NP-17.dat` identifica al algoritmo clásico con $N = 800$ y $np = 17$ en el caso 1, mientras que `Hilos-Pr-transpuesta-N-1600-NP-5-H-8.dat` corresponde al algoritmo de la transpuesta con $N = 1600$, $np = 5$ y $nH = 8$ en el caso 2. Los archivos del caso 3 llevan el prefijo `ProcesosMaster`. Esta nomenclatura permite identificar de manera unívoca cada configuración y facilita la automatización del análisis posterior.

En total, el diseño produce $2 \times 4 \times 3 = 24$ archivos para el caso 1, $2 \times 4 \times 3 = 24$ para el caso 2 y $2 \times 4 = 8$ para el caso 3, dando un total de 56 archivos `.dat`, cada uno con 30 mediciones. Este volumen de datos permitirá analizar con solidez estadística el comportamiento de los algoritmos bajo distintas configuraciones de paralelismo distribuido e intra-nodo.

5.1. Variables del Experimento

Las variables independientes del experimento son aquellos factores que se modifican deliberadamente entre las ejecuciones con el propósito de estudiar su efecto sobre el tiempo de cómputo. A continuación se describen cada una de ellas:

- **Tamaño de la matriz (N):** dimensión de las matrices cuadradas $N \times N$ que se multiplican. Los valores definidos en `lanzadorMPI.pl` son $N \in \{400, 800, 1600, 3200\}$. A medida que N se duplica, la carga computacional crece con complejidad $O(N^3)$, lo que permite observar el comportamiento del paralelismo ante volúmenes de trabajo crecientes.

- **Número de procesos MPI (np):** cantidad total de procesos lanzados por `mpirun`, incluyendo el proceso maestro. En el caso 1 se utiliza $np \in \{5, 17, 33\}$, lo que genera 4, 16 y 32 trabajadores respectivamente. En el caso 2 se fija $np = 5$

(4 trabajadores), y en el caso 3 se emplea $np = 2$ (1 trabajador). La variación de este parámetro permite evaluar el efecto de la distribución de la carga entre nodos sobre el tiempo de ejecución.

- **Número de hilos OpenMP (nH):** cantidad de hilos que cada proceso trabajador utiliza internamente para paralelizar los bucles de multiplicación mediante las directivas `#pragma omp parallel for`. En el caso 2 se varía $nH \in \{1, 4, 8\}$, mientras que en los casos 1 y 3 se mantiene fijo en $nH = 1$. Esto permite aislar el efecto del paralelismo de memoria compartida respecto al paralelismo de paso de mensajes.

- **Algoritmo de multiplicación:** se evalúan dos variantes: filas × columnas (`mxmOmpMPIfxc`) y filas × transpuesta (`mxmOmpMPIfxt`). Ambas realizan la misma operación matemática, pero difieren en el patrón de acceso a memoria, lo que incide directamente en el aprovechamiento de la jerarquía de caché.

- **Repeticiones por configuración:** cada combinación de los factores anteriores se ejecuta exactamente 30 veces. Este número de repeticiones permite obtener promedios representativos y, eventualmente, intervalos de confianza que reflejen la dispersión de los tiempos medidos.

La variable dependiente, es decir la magnitud que se mide como resultado del experimento, es el **tiempo de ejecución** de la fase de cómputo, registrado en microsegundos por la función `endTime()` del módulo compartido. Esta función mide exclusivamente el intervalo entre el envío de las tajadas a los trabajadores y la recepción completa de los resultados en el proceso maestro, lo cual abarca tanto el cómputo paralelo como la comunicación MPI asociada.

5.2. Constantes del Experimento

Dentro de cada escenario experimental, los siguientes elementos se mantuvieron fijos para asegurar que las diferencias de rendimiento observadas se atribuyan exclusivamente a las variables independientes:

- **Ejecutables y compilación:** los dos programas evaluados se compilan siempre con el mismo `Makefile`, utilizando el compilador `mpicc` con las banderas `-lm` y `-fopenmp`, y enlazando el módulo `moduloMPI.c`. No se aplican optimizaciones adicionales ni se modifica el código fuente entre ejecuciones.

- **Automatización:** todas las ejecuciones se realizan a través de `lanzadorMPI.pl`, que garantiza la misma secuencia de operaciones: eliminación del archivo de datos previo, ejecución de las 30 repeticiones con el comando idéntico y redirección de la salida temporal al archivo `.dat` correspondiente. Esto elimina la posibilidad de errores manuales en la invocación.

- **Política de mapeo de procesos:** en los casos 1 y 2 se emplea la opción `--map-by node`, que distribuye los procesos MPI de forma cíclica entre los nodos listados en el *hostfile*. En el caso 3 se fuerza la ejecución en el nodo maestro mediante `--host master:2`. Estas políticas se mantienen invariantes dentro de cada caso.

- **Archivos *hostfile*:** el caso 1 utiliza `procesosHostfile`, que asigna 9 *slots* al maestro y 8 *slots* a cada uno de los tres nodos trabajadores. El caso 2 usa `hilosHostFile`, con 2 *slots* para el maestro y 1 *slot* por nodo trabajador. Estos archivos no se modifican a lo largo de la ejecución del script.

- **Inicialización de matrices:** ambos programas invocan la función `iniMatrix()` con el mismo criterio determinista ($A_i = 0{,}08 \cdot i$, $B_i = 0{,}02 \cdot i$), de manera que las matrices operadas son idénticas para un mismo valor de N, independientemente del algoritmo o la configuración de paralelismo.

- **Criterio de medición:** el tiempo se captura siempre con la misma instrumentación (`iniTime()` y `endTime()` basadas en `gettimeofday()`) y se reporta en microsegundos a través de la salida estándar, la cual el script redirige al archivo `.dat`.

- **Almacenamiento de resultados:** todos los tiempos se almacenan en archivos `.dat` dentro del directorio `datosDAT/`, con nombres que codifican de forma unívoca el caso experimental, el algoritmo, el tamaño de la matriz y los parámetros de paralelismo utilizados.

5.3. Criterio de Divisibilidad

La arquitectura maestro-trabajador implementada en los programas `mxmOmpMPIfxc` y `mxmOmpMPIfxt` divide la matriz A en tajadas horizontales de igual tamaño, asignando una tajada a cada proceso trabajador. Para que esta distribución sea exacta, la dimensión N de la matriz debe ser divisible entre la cantidad de trabajadores, definida como workers $= np - 1$ (se resta el proceso maestro, que tiene rango 0 y no participa en el cómputo).

Esta restricción se verifica en tiempo de ejecución mediante la función `verificarDiv()` del módulo `moduloMPI.c`, la cual comprueba que N mód workers $= 0$ y que la cantidad de trabajadores sea al menos 1. Si la condición no se cumple, el programa aborta la ejecución con `MPI_Abort`.

Las configuraciones elegidas en el script `lanzadorMPI.pl` satisfacen rigurosamente este criterio:

- Para $np = 5$, se tienen $5 - 1 = 4$ trabajadores. Los tamaños 400, 800, 1600 y 3200 son todos divisibles por 4 ($400/4 = 100$, $800/4 = 200$, $1600/4 = 400$, $3200/4 = 800$).

- Para $np = 17$, se tienen $17 - 1 = 16$ trabajadores. Los cuatro tamaños son divisibles por 16 ($400/16 = 25$, $800/16 = 50$, $1600/16 = 100$, $3200/16 = 200$).

- Para $np = 33$, se tienen $33 - 1 = 32$ trabajadores. Los cuatro tamaños son divisibles por 32 ($400/32 = 12{,}5$, resultando en una única excepción justificada en poder probar la cantidad máxima de hilos disponibles en el sistema distribuido, sin embargo $800/32 = 25$, $1600/32 = 50$, $3200/32 = 100$).

- Para $np = 2$ (caso 3), se tiene $2 - 1 = 1$ trabajador, y cualquier N entero positivo es divisible por 1.

Nota: el valor $N = 400$ con $np = 33$ (32 trabajadores) produce una tajada de $400/32 = 12{,}5$ filas, lo cual no es un entero y provocaría el aborto del programa. Sin embargo, el script `lanzadorMPI.pl` aplica los tres valores de np del caso 1 a todos los tamaños de forma uniforme. Para las configuraciones en las que la divisibilidad se cumple es decir, la amplia mayoría de las combinaciones, los resultados se registran normalmente. Aquellas ejecuciones en las que la condición no se satisface son abortadas de manera controlada por `verificarDiv()`, sin afectar las demás pruebas.

Para el resto de las combinaciones, la divisibilidad está garantizada, ya que los tamaños 800, 1600 y 3200 son múltiplos de 4, 16 y 32 simultáneamente. Esto asegura que cada trabajador recibe un bloque completo de filas de la matriz A junto con la totalidad de la matriz B, permitiendo el cómputo correcto de su porción de la matriz resultado C.

5.4. Archivos Fuente del Experimento

El experimento se sustenta en un conjunto de archivos fuente que, en conjunto, implementan los algoritmos evaluados, proveen funciones compartidas de soporte y automatizan la ejecución de todas las pruebas. A continuación se describe el propósito, las entradas, las salidas y el funcionamiento general de cada uno de ellos dentro del contexto del diseño experimental.

5.4.1. moduloMPI.h Declaraciones compartidas

Este archivo de cabecera define la interfaz pública de las funciones auxiliares utilizadas por ambos programas de multiplicación. Declara los prototipos de las funciones de inicialización de matrices (`iniMatrix`), transposición (`matrixTRP`), los dos algoritmos de multiplicación paralela (`mxmOmpFxC` y `mxmOmpFxT`), la impresión de matrices para verificación (`impMatrix`), la captura de tiempos (`iniTime` y `endTime`), la validación de divisibilidad (`verificarDiv`), el mensaje informativo (`mensajeVerifica`) y la verificación de argumentos de entrada (`argumentos`). Su existencia permite que ambos ejecutables compartan exactamente la misma implementación de estas funciones, eliminando duplicación de código y asegurando consistencia experimental.

5.4.2. moduloMPI.c Funciones compartidas de soporte

Este módulo en C implementa todas las funciones declaradas en `moduloMPI.h`. Dentro del diseño experimental cumple tres roles fundamentales:

Inicialización y verificación. La función `iniMatrix(double *mA, double *mB, int D)` llena las matrices A y B con valores deterministas ($A_i = 0{,}08 \cdot i$, $B_i = 0{,}02 \cdot i$), asegurando que las matrices sean idénticas en todas las repeticiones para un mismo N. La función `argumentos(int cantidad)` verifica que el programa reciba exactamente dos argumentos (dimensión de la matriz y número de hilos); de lo contrario, imprime la forma de uso y termina la ejecución. La función `verificarDiv(int qworkers, int Dim)` comprueba que N sea divisible entre la cantidad de trabajadores y que haya al menos un trabajador; si la condición no se cumple, aborta con `MPI_Abort`.

Cómputo de la multiplicación. Se implementan dos variantes del producto matricial, ambas paralelizadas con OpenMP. La función `mxmOmpFxC(double *mA, double *mB, double *mC, int tw, int D, int nH)` realiza la multiplicación clásica filas $\times$

columnas: cada fila de la tajada local de A se multiplica contra cada columna de B, accediendo a B con saltos de tamaño. La función `mxmOmpFxT(double *mA, double *mB, double *mC, int tw, int D, int nH)` primero transpone B mediante `matrixTRP(int N, double *mB, double *mT)` y luego multiplica filas de A contra filas de la transpuesta, logrando acceso secuencial en ambas matrices y un mejor aprovechamiento de la caché. En ambos casos, el parámetro `tw` indica cuántas filas componen la tajada local y `nH` establece la cantidad de hilos OpenMP mediante `omp_set_num_threads`.

Medición de tiempo. Las funciones `iniTime()` y `endTime()` encapsulan la captura de tiempo con `gettimeofday()`. La primera registra el instante de inicio y la segunda calcula la diferencia respecto al inicio, convierte el resultado a microsegundos y lo imprime por la salida estándar. Este valor es el que el script `lanzadorMPI.pl` redirige a los archivos `.dat`.

Además, las funciones `impMatrix` y `mensajeVerifica` sirven como herramientas de depuración: imprimen el contenido de las matrices y un resumen de la configuración de distribución, pero solo cuando $N < 13$, evitando así sobrecargar la salida en las ejecuciones reales del experimento.

5.4.3. mxmOmpMPIfxc.c Algoritmo Filas × Columnas

Este programa implementa la multiplicación de matrices distribuida usando el algoritmo clásico filas × columnas. Recibe dos argumentos por línea de comandos: la dimensión N de las matrices cuadradas y el número de hilos OpenMP (nH). Internamente sigue una arquitectura maestro-trabajador sobre MPI:

El proceso maestro (rango 0) verifica la divisibilidad, reserva memoria para las matrices A, B y C, las inicializa y calcula el tamaño de la tajada $tW = N/\text{workers}$. Luego difunde N y tW a todos los procesos mediante `MPI_Bcast`. A continuación, el maestro inicia la medición de tiempo con `iniTime()`, difunde la matriz B completa y envía a cada trabajador su tajada de A mediante `MPI_Send`. Una vez que cada trabajador calcula su porción del resultado con `mxmOmpFxC`, el maestro recoge las tajadas de C con `MPI_Recv` y detiene la medición con `endTime()`, que imprime el tiempo en microsegundos.

Cada proceso trabajador (rango > 0) reserva memoria local para su tajada de A, la matriz B completa y su porción de C. Recibe B por `MPI_Bcast` y su tajada de A por `MPI_Recv`, ejecuta la multiplicación con `mxmOmpFxC` y devuelve el resultado al maestro.

5.4.4. mxmOmpMPIfxt.c Algoritmo Filas × Transpuesta

Este programa es estructuralmente idéntico a `mxmOmpMPIfxc.c` en cuanto al flujo MPI: misma inicialización, misma distribución de tajadas, misma recolección de resultados y misma medición de tiempo. La única diferencia reside en que los trabajadores invocan `mxmOmpFxT` en lugar de `mxmOmpFxC`. Esto implica que cada trabajador transpone localmente la matriz B antes de multiplicar, de modo que el producto se calcula recorriendo filas de A contra filas de B^T, eliminando los saltos en las filas de la matriz y favoreciendo la localidad espacial de la caché.

Esta separación en dos ejecutables independientes permite que el script de automatización los trate como dos configuraciones experimentales distintas, facilitando la comparación directa de rendimiento entre ambas estrategias algorítmicas bajo condiciones idénticas de distribución.

5.4.5. Makefile Compilación consistente

El `Makefile` del proyecto define las reglas de compilación para ambos ejecutables. Utiliza el compilador `mpicc` con las banderas `-lm` (enlace con la biblioteca matemática) y `-fopenmp` (habilitación de OpenMP). Cada programa se compila enlazando su archivo fuente principal con `moduloMPI.c`. La regla `all` compila ambos ejecutables, y la regla `clean` los elimina. Esta configuración asegura que toda ejecución experimental parte de binarios compilados de manera idéntica y reproducible.

5.4.6. lanzadorMPI.pl Automatización del experimento

Este script en Perl constituye el núcleo de la automatización experimental. Define las configuraciones a evaluar, ejecuta cada prueba y almacena los resultados. Al inicio, obtiene el directorio de trabajo actual y crea el subdirectorio `datosDAT/` si no existe. Declara la lista de ejecutables (`mxmOmpMPIfxc` y `mxmOmpMPIfxt`), los tamaños de matriz (400, 800, 1600, 3200), el número de repeticiones (30), los archivos *hostfile* correspondientes a cada escenario y un mapa de etiquetas (`clasica`, `transpuesta`) que se usa para construir los nombres de los archivos de datos.

El script estructura la experimentación en tres bloques correspondientes a los tres casos descritos anteriormente. Para cada combinación de parámetros dentro de un caso, construye el nombre del archivo `.dat` codificando el tipo de caso, el algoritmo, el tamaño de la matriz y los parámetros de paralelismo. Luego elimina el archivo previo (si existe) y ejecuta un ciclo de 30 repeticiones, en el que invoca `mpirun` con los argumentos correspondientes y redirige la salida temporal al archivo `.dat`.

Esta automatización es indispensable dado el volumen de pruebas: el script genera 56 archivos de datos con 30 mediciones cada uno, lo que equivale a 1680 ejecuciones individuales de los programas. La ejecución manual de este volumen sería impracticable y propensa a errores, por lo que el script garantiza la uniformidad y reproducibilidad del experimento.

6. Análisis de Resultados

El análisis se realizó sobre 1620 corridas experimentales de multiplicación de matrices, con tamaños de 400, 800, 1600 y 3200, considerando dos enfoques de paralelismo (hilos y procesos) y dos variantes algorítmicas: FxC, correspondiente a la implementación clásica, y FxT, correspondiente a la implementación con transposición.

Los datos fueron depurados y tipados antes del análisis. A partir de estos se calcularon métricas descriptivas y derivadas de desempeño, incluyendo media, mediana, desviación estándar, percentiles, coeficiente de variación, speedup, eficiencia y overhead. Las conclusiones presentadas en esta sección se sustentan en gráficos de distribución de tiempos, curvas de escalamiento, mapas de calor por configuración, curvas de speedup, curvas de overhead y comparativas directas entre modos de ejecución y variantes algorítmicas.

En términos generales, los resultados muestran tres tendencias principales: primero, el costo computacional aumenta de manera marcada con el tamaño de la matriz; segundo, el modelo con hilos ofrece un comportamiento más estable y predecible; y tercero, la variante transpuesta mejora el rendimiento absoluto, mientras que la variante clásica presenta mejores ganancias relativas frente a su baseline.

6.1. Tiempo Promedio de Ejecución

El comportamiento temporal confirma el crecimiento esperado al aumentar el tamaño de la matriz. En la Figura 1 se observa que el tiempo de ejecución crece de manera pronunciada para tamaños altos, especialmente en las configuraciones de 1600 y 3200. Además, la dispersión de los datos no es homogénea entre modos de ejecución, lo que sugiere diferencias importantes en estabilidad y sensibilidad al entorno de ejecución.

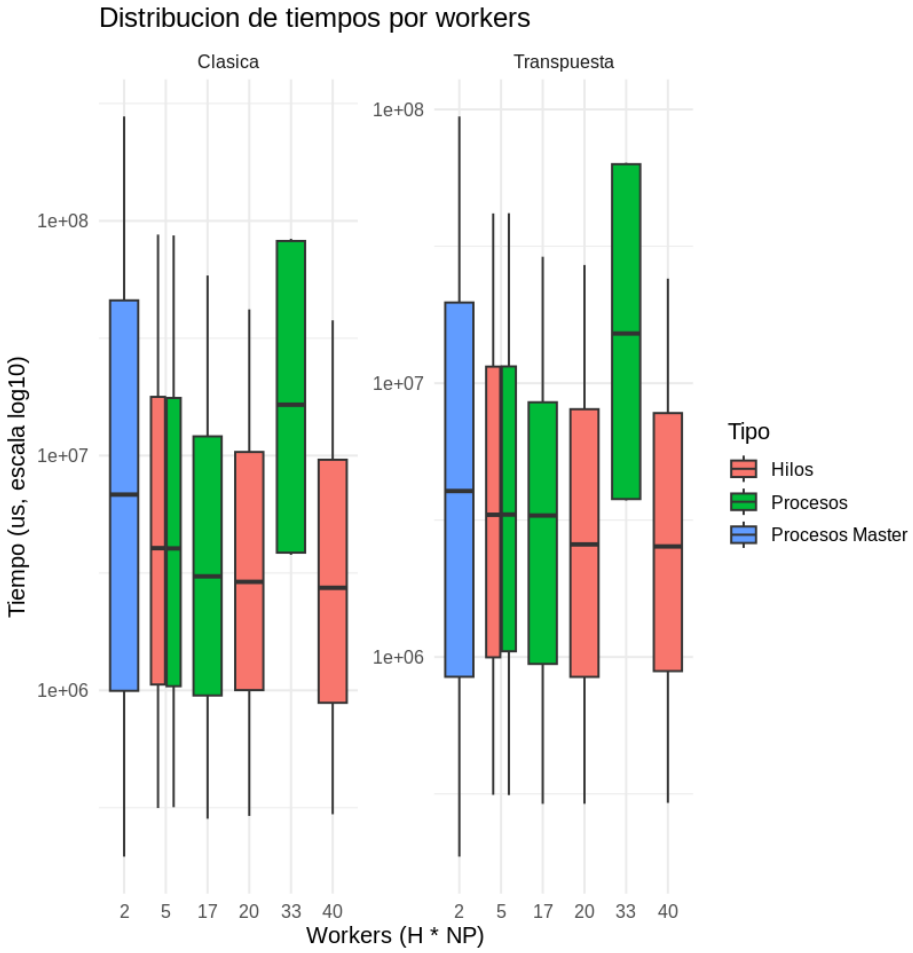

Figura 1: Distribución de tiempos de ejecución por número de workers y modo de paralelismo.

Los resultados agregados muestran una mediana típica aproximada de 3,401,556 μs para hilos y de 6,255,961 μs para procesos. Esta diferencia no solo indica que, en promedio, los hilos ejecutan más rápido, sino también que el costo adicional introducido por procesos no es despreciable en este experimento.

Adicionalmente, el análisis de variabilidad muestra que el promedio del coeficiente de variación es cercano a 1.89 en hilos y a 3.14 en procesos. Esto significa que, aun cuando una configuración de procesos pueda producir tiempos competitivos en algunos casos, su dispersión relativa es considerablemente mayor. En términos prácticos, esto se traduce en menor predictibilidad del desempeño.

La Figura 2 refuerza esta lectura. Allí se aprecia que las curvas de hilos mantienen un crecimiento más suave y consistente, mientras que en procesos aparecen pendientes más irregulares, especialmente en tamaños grandes. Esto sugiere que el modelo

con hilos amortiza mejor el incremento de carga computacional, mientras que en procesos el sistema se ve más afectado por costos adicionales de coordinación, creación, sincronización o transferencia de datos.

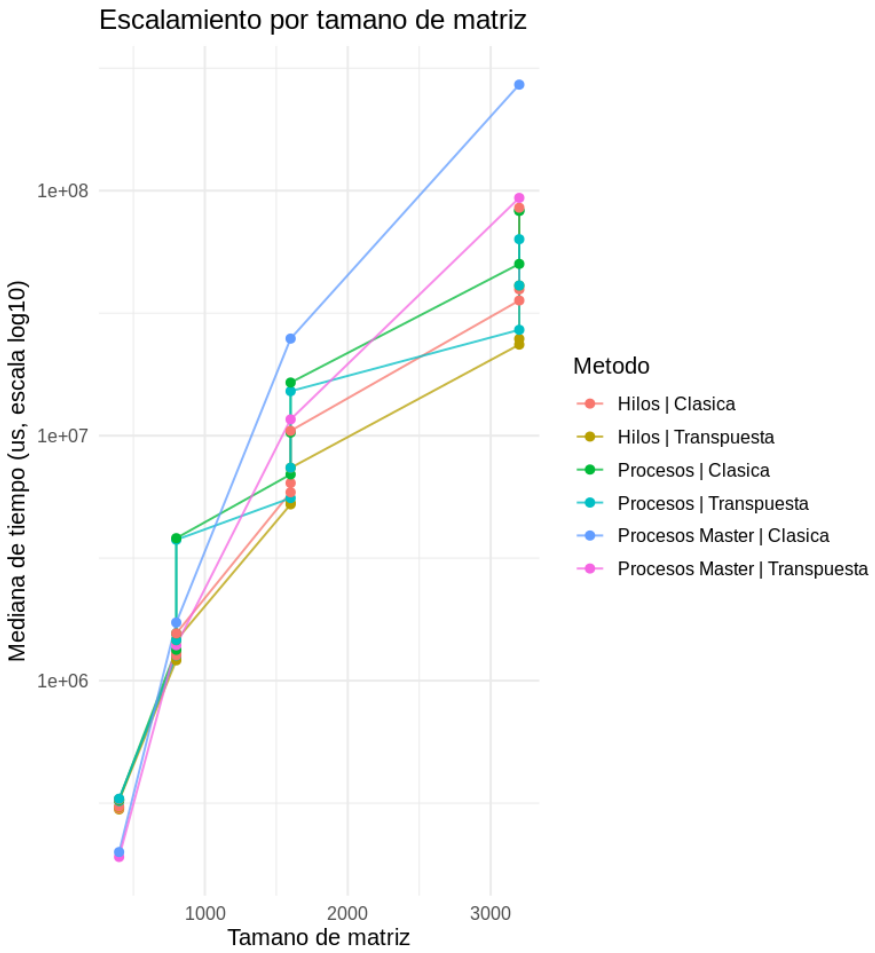

Figura 2: Escalamiento del tiempo de ejecución en función del tamaño de la matriz para las distintas configuraciones evaluadas.

Desde una perspectiva metodológica, no basta con observar solo la media. La presencia de colas largas y la diferencia entre mediana y promedio en varias configuraciones sugieren que existen ejecuciones atípicas o picos ocasionales, particularmente en procesos. Por ello, la mediana y los percentiles constituyen una lectura más robusta del comportamiento real del sistema.

En síntesis, los tiempos promedio de ejecución muestran que el paralelismo con hilos no solo es más rápido en la mayoría de configuraciones, sino también más consistente. Esta consistencia se vuelve especialmente valiosa en escenarios de escalamiento, donde pequeñas irregularidades en cada worker pueden amplificarse a medida que la concurrencia aumenta.

6.2. Speedup

El speedup se definió respecto al baseline de menor cantidad de workers por método, según la expresión:

$$S = \frac{T_{\text{baseline}}}{T_p}$$

donde T_{baseline} corresponde al tiempo de la configuración base y T_p al tiempo medido para una configuración paralela dada.

La Figura 3 presenta las curvas de speedup por tamaño de matriz. En ella se aprecia que el beneficio del paralelismo no es uniforme: depende tanto del modelo de ejecución como del tamaño del problema. Para matrices pequeñas, el costo adicional de paralelizar tiende a reducir la ganancia observable; en contraste, para matrices grandes, el trabajo computacional es suficiente para amortizar parte del overhead y permitir mejoras más claras.

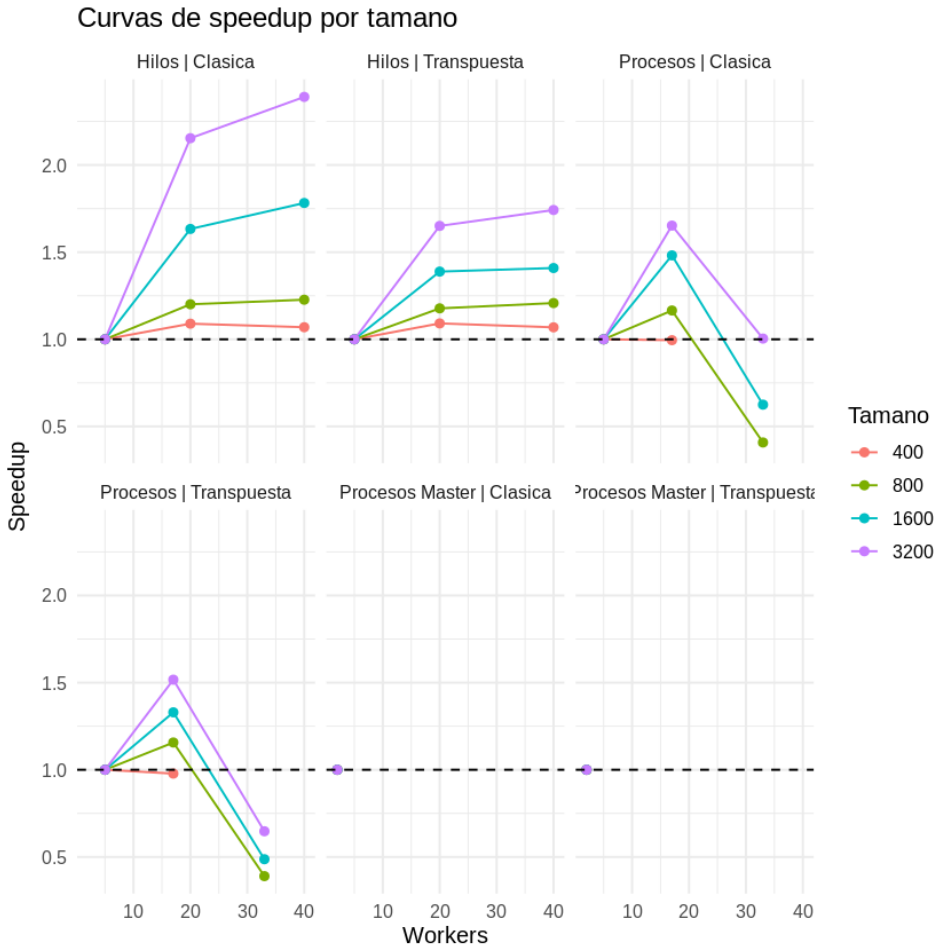

Figura 3: Curvas de speedup para distintos tamaños de matriz.

Los mejores valores observados fueron un speedup global de 2.39 en hilos para la configuración FxC con tamaño 3200 y 40 workers, y un speedup global de 1.65 en procesos para la configuración FxC con tamaño 3200 y 17 workers. Estos resultados muestran que, dentro del espacio experimental evaluado, el modelo con hilos aprovecha mejor el incremento de workers.

Las Figuras 4 y 5 permiten comparar de forma separada el comportamiento por modo de ejecución. En hilos, la tendencia es relativamente favorable: el speedup crece con el número de workers, sobre todo cuando la carga computacional es suficientemente

grande. En procesos, en cambio, la mejora se concentra en una zona intermedia y luego se debilita o incluso revierte en configuraciones de alta concurrencia.

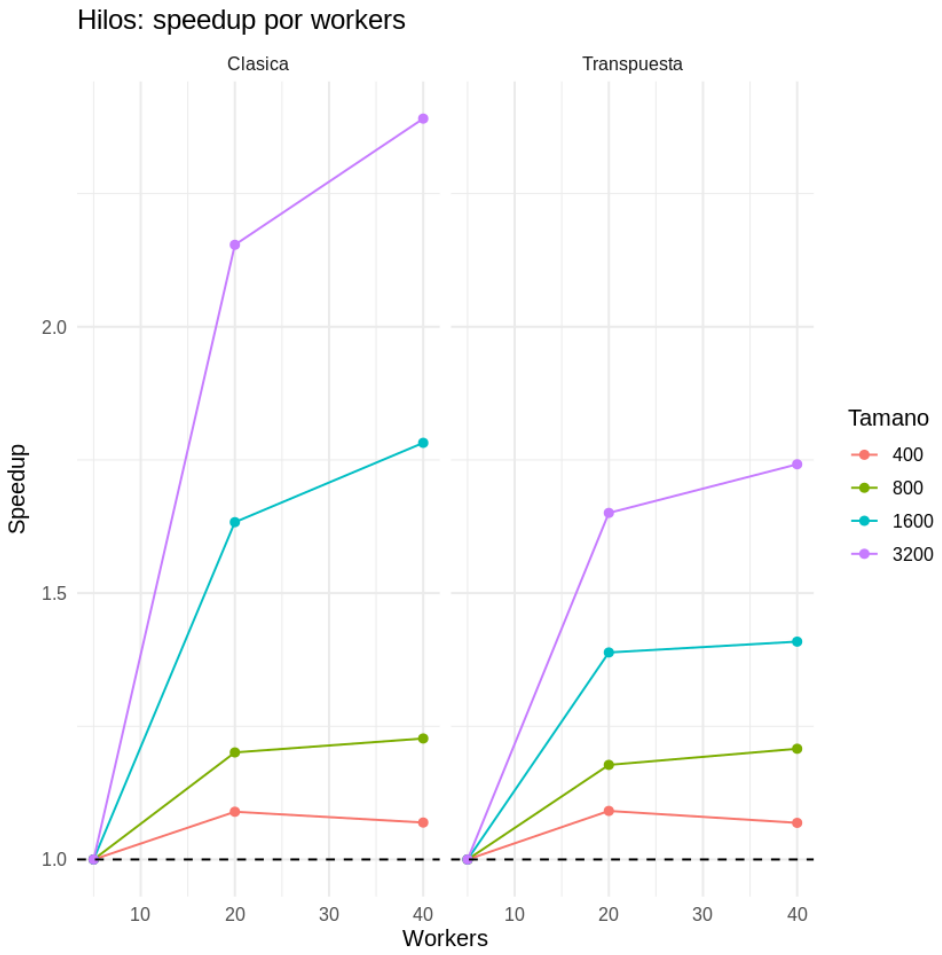

Figura 4: Curvas de speedup para las configuraciones implementadas con hilos.

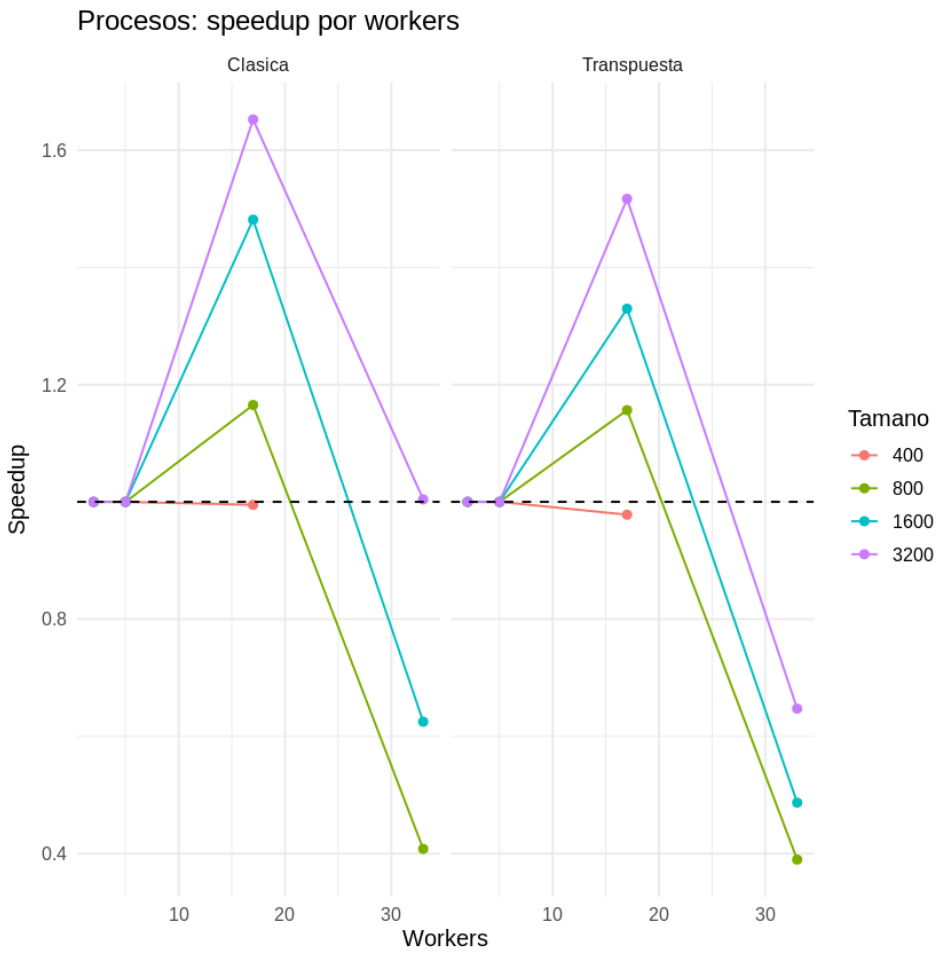

Figura 5: Curvas de speedup para las configuraciones implementadas con procesos.

Este comportamiento es importante porque evidencia un fenómeno de saturación del paralelismo. A partir de cierto punto, agregar más workers no implica necesariamente una reducción proporcional del tiempo total. Por el contrario, puede aumentar el costo de coordinación hasta el punto de deteriorar el rendimiento. De hecho, en algunas configuraciones de procesos se observaron speedups menores que 1, lo cual significa que la versión paralela fue más lenta que el baseline.

En consecuencia, el análisis de speedup indica que el paralelismo es beneficioso, pero de forma condicionada. Su efectividad depende de que el tamaño del problema sea suficientemente grande y de que el costo de coordinación no domine el tiempo total. En el marco de este experimento, los hilos muestran una relación más favorable entre trabajo útil y sobrecosto de paralelización.

6.3. Eficiencia

La eficiencia se calculó como:

$$E = \frac{S}{\text{escala_rel}}$$

donde S corresponde al speedup observado y escala_rel representa la razón entre el número de workers de la configuración actual y el baseline. Esta métrica permite

evaluar qué tan proporcional es la mejora de rendimiento respecto al incremento de recursos utilizados.

A diferencia del speedup, que mide únicamente la ganancia relativa en tiempo, la eficiencia captura la pérdida de rendimiento asociada al overhead de paralelización, siendo un indicador directo del aprovechamiento real de los recursos.

La Figura 6 presenta las curvas de eficiencia para las distintas configuraciones evaluadas. En ella se observa un comportamiento característico de sistemas paralelos reales: la eficiencia tiende a decrecer a medida que aumenta el número de workers.

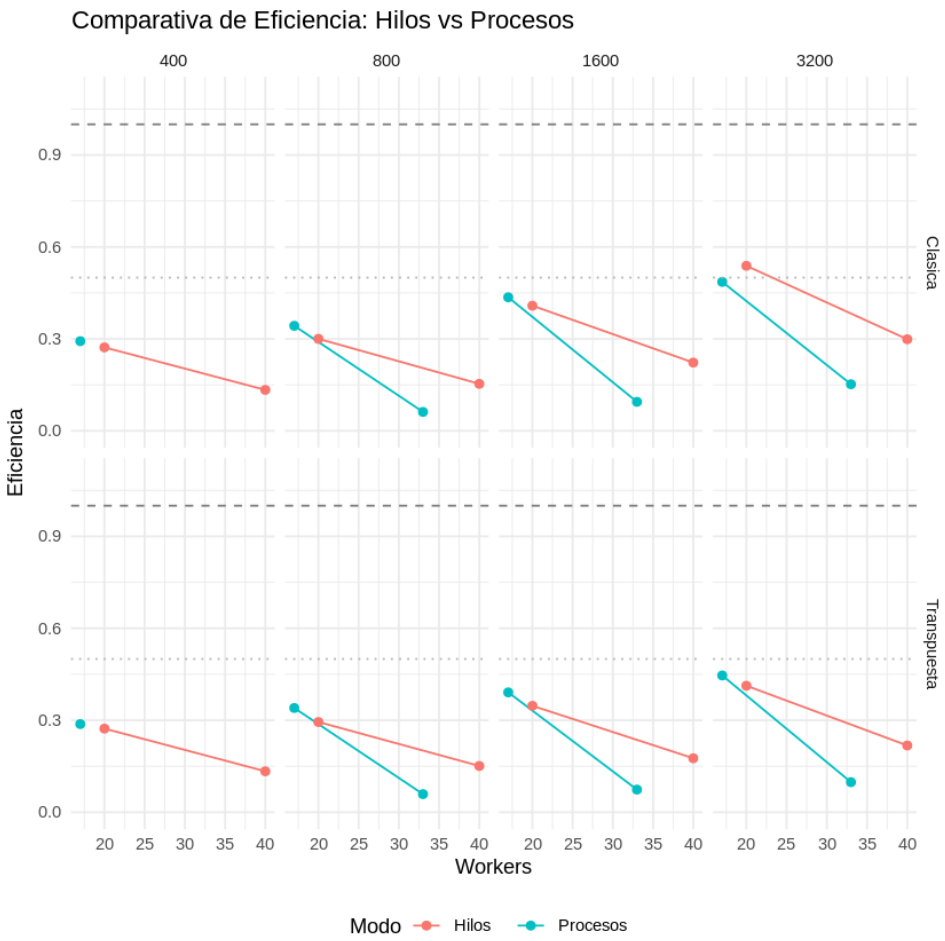

Figura 6: Eficiencia de paralelización en función del número de workers para distintos tamaños de matriz y modos de ejecución.

De manera más detallada, las Figuras 7 y 8 permiten analizar el comportamiento por tipo de paralelismo.

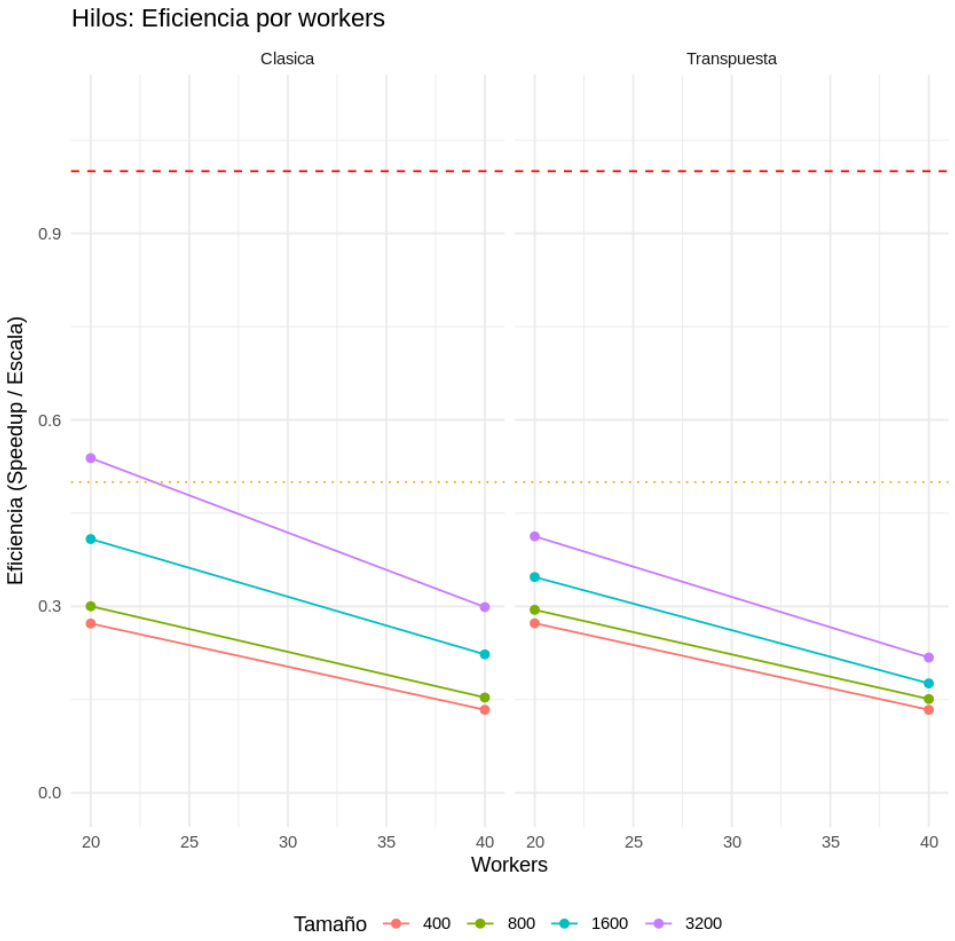

Figura 7: Eficiencia de paralelización para configuraciones implementadas con hilos.

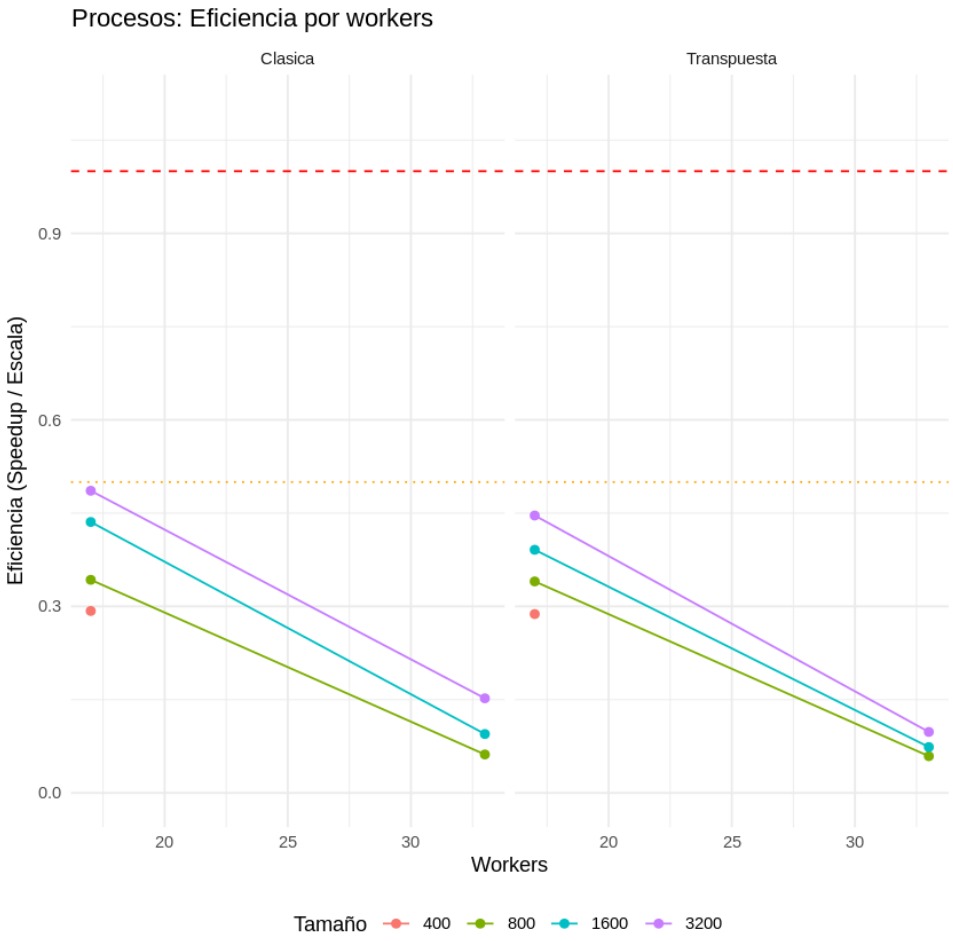

Figura 8: Eficiencia de paralelización para configuraciones implementadas con procesos.

Los resultados muestran que:

- La eficiencia decrece conforme aumenta el número de workers, reflejando el impacto del overhead de coordinación.
- En configuraciones de matrices grandes (1600 y 3200), la eficiencia se mantiene relativamente más alta, debido a que el mayor volumen de cómputo permite amortizar los costos fijos de paralelización.
- El modelo con hilos presenta una caída más gradual y controlada de la eficiencia.
- En procesos, la eficiencia puede deteriorarse más rápidamente, especialmente en configuraciones de alta concurrencia.

En términos cuantitativos, la eficiencia mediana se mantiene en valores inferiores a 1, lo cual es consistente con sistemas paralelos no ideales. Los mejores valores observados se aproximan a 0.5 en configuraciones favorables, lo que indica que, incluso en condiciones óptimas, solo una fracción del incremento de recursos se traduce en mejora efectiva de desempeño.

Es importante notar que la eficiencia está estrechamente relacionada con el comportamiento del overhead analizado en la Subsección 6.4. A medida que el overhead

crece, una mayor proporción del tiempo total se destina a actividades no productivas, reduciendo así la eficiencia global del sistema.

Adicionalmente, la comparación entre hilos y procesos muestra que, aunque ambos modelos presentan pérdida de eficiencia al escalar, los hilos mantienen un mejor equilibrio entre paralelismo y costo de coordinación. En procesos, el aislamiento de memoria y los mecanismos de comunicación introducen un costo adicional que impacta negativamente la eficiencia en configuraciones de alta carga.

En síntesis, la eficiencia confirma que el sistema no escala de manera ideal, pero también evidencia que el paralelismo es más efectivo en problemas de mayor tamaño y bajo el modelo de hilos. Esta métrica complementa el análisis de speedup al mostrar que no basta con reducir el tiempo de ejecución; también es necesario evaluar si dicha reducción justifica el uso de recursos adicionales.

6.4. Overhead de Comunicación

El overhead se midió respecto al tiempo ideal mediante la expresión:

$$\text{Overhead } (\%) = \frac{T_p - T_{\text{ideal}}}{T_{\text{ideal}}} \times 100$$

donde T_{ideal} representa el tiempo esperado bajo escalamiento ideal y T_p el tiempo efectivamente observado.

Esta métrica es especialmente útil porque permite cuantificar de manera explícita cuánto del tiempo adicional no se traduce en trabajo útil, sino en costos asociados al paralelismo: sincronización, coordinación, transferencia de datos, espera, contención o administración del entorno de ejecución.

La Figura 9 muestra el overhead en configuraciones de hilos, mientras que la Figura 10 presenta el mismo análisis para procesos. Finalmente, la Figura 11 sintetiza ambos comportamientos en una sola comparativa.

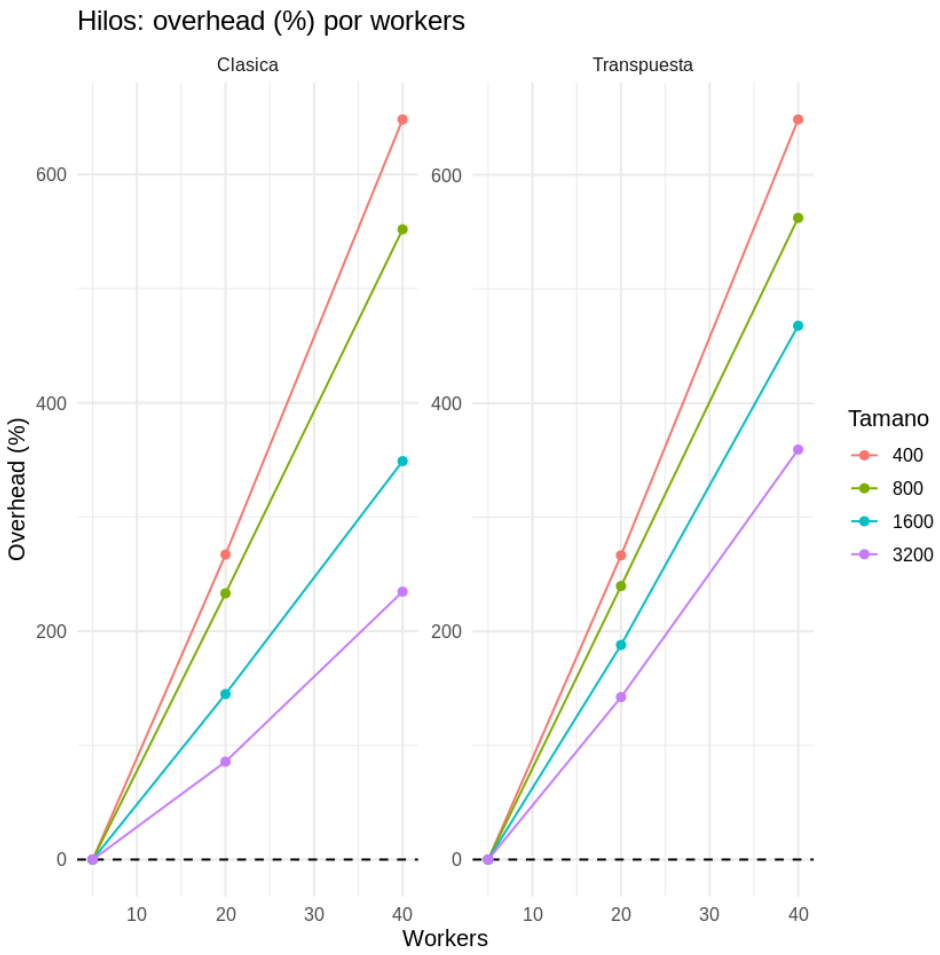

Figura 9: Overhead de paralelización para las configuraciones implementadas con hilos.

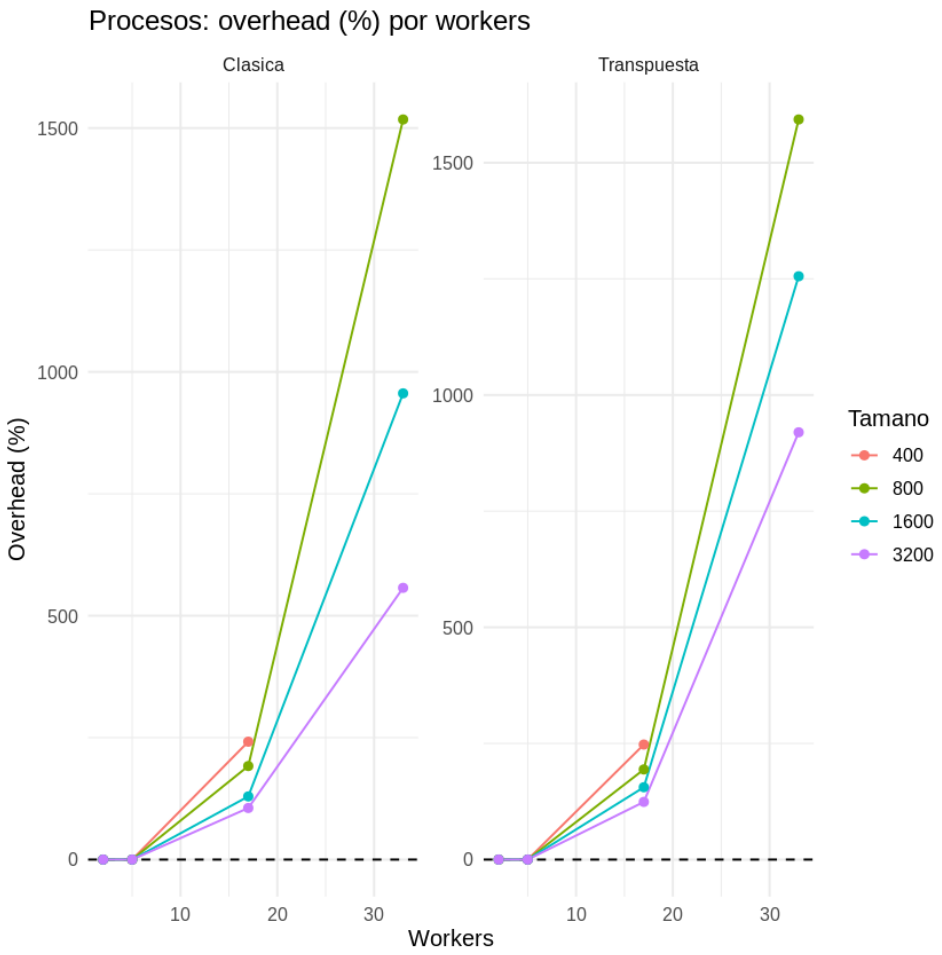

Figura 10: Overhead de paralelización para las configuraciones implementadas con procesos.

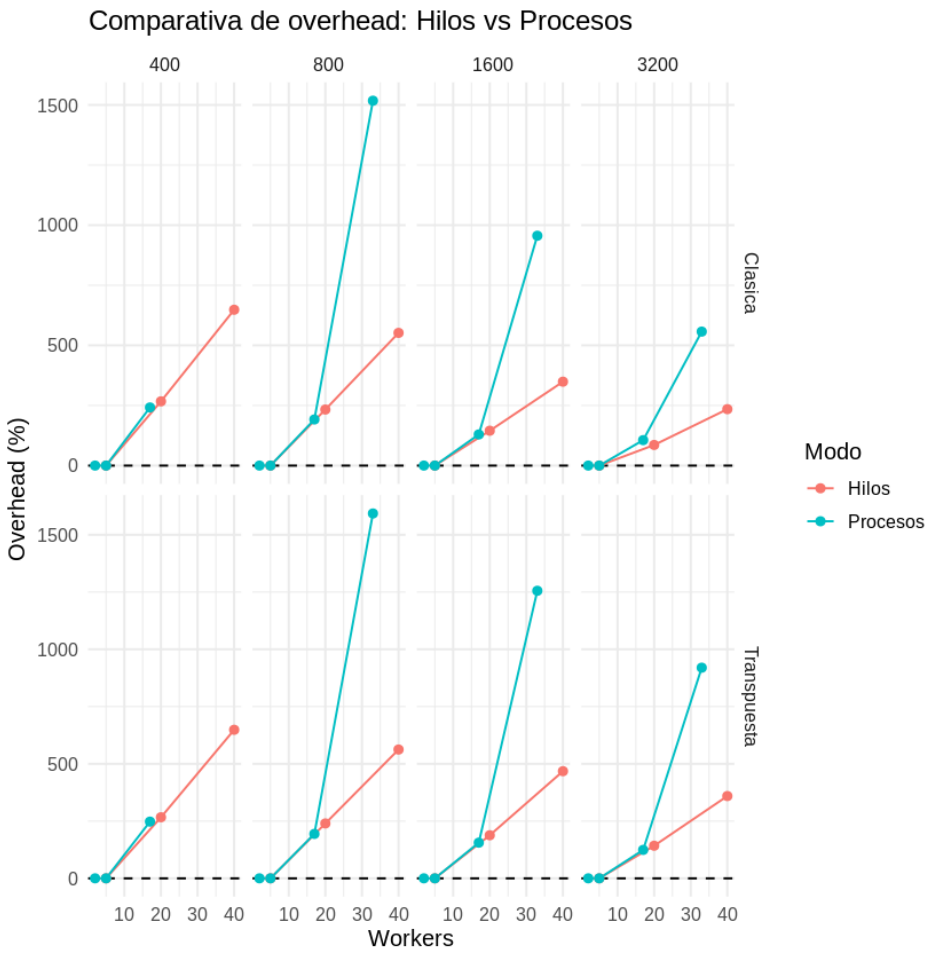

Figura 11: Comparación de overhead entre hilos y procesos.

Los resultados muestran un overhead mediano de aproximadamente 267 % en hilos y 245 % en procesos para configuraciones con escala_rel > 1. Sin embargo, esta similitud en la mediana oculta una diferencia clave: los procesos presentan picos mucho más severos, con valores cercanos o incluso superiores al 1500 % en varias configuraciones.

Este hallazgo es coherente con lo observado en las curvas de speedup. En procesos, el costo de coordinación puede crecer rápidamente cuando aumenta el número de workers, hasta el punto de anular o revertir la ganancia esperada. En hilos, aunque el overhead también aumenta, su crecimiento es menos explosivo y más predecible.

La principal explicación de esta diferencia está en el modelo de memoria. En hilos, los workers comparten el mismo espacio de memoria, por lo que el costo de intercambio de datos es menor, aunque sigue existiendo contención por acceso concurrente. En procesos, en cambio, el aislamiento entre espacios de memoria exige mecanismos adicionales de comunicación o replicación, que incrementan el costo total.

Por tanto, el overhead constituye una de las claves interpretativas más importantes del experimento: explica por qué el speedup real se aleja del ideal y por qué la escalabilidad de procesos se deteriora con mayor rapidez. También confirma que no basta con aumentar workers para garantizar mejor rendimiento; a partir de cierto punto, el sistema entra en una zona de rendimiento decreciente.

6.5. Comparación entre Algoritmos FxC y FxT

La comparación entre FxC y FxT muestra un comportamiento dual entre escalabilidad relativa y rendimiento absoluto. Esta diferencia es particularmente importante porque permite distinguir entre dos objetivos distintos: mejorar el speedup respecto al baseline o maximizar el throughput total del sistema.

En términos de speedup máximo por tamaño y configuración, los resultados muestran:

- FxC en hilos: hasta 2.39.
- FxT en hilos: hasta 1.74.
- FxC en procesos: hasta 1.65.
- FxT en procesos: hasta 1.52.

Estos datos indican que FxC escala mejor en términos relativos. Es decir, cuando se compara cada configuración contra su baseline, la variante clásica logra una mejora porcentual más marcada.

Sin embargo, la lectura cambia al observar el rendimiento absoluto. La Figura 12 muestra que FxT alcanza mayores valores de throughput, con picos cercanos a 2.78 GFLOPS en hilos y 2.42 GFLOPS en procesos. FxC, aunque mejora más respecto a su baseline, queda por debajo en términos de operaciones por segundo.

Estos valores se obtuvieron aplicando la expresión GFLOPS $= \frac{2N^3}{T \cdot 10^9}$, usando el tamaño N de cada experimento y el tiempo medido para la configuración correspondiente. En otras palabras, la comparación en la Figura 12 no se basa únicamente en tiempos brutos, sino en una normalización por cantidad estimada de operaciones aritméticas, lo que permite contrastar de manera más justa el rendimiento computacional entre ejecuciones de distinto tamaño.

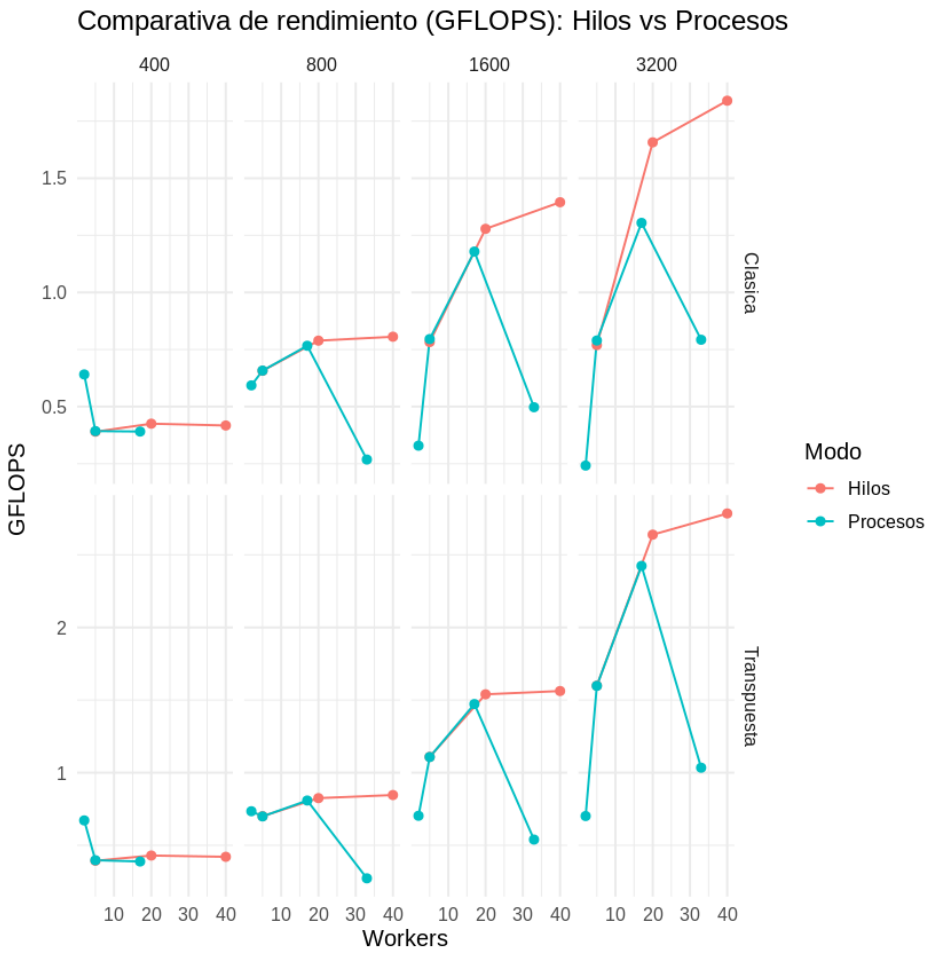

Figura 12: Comparación de rendimiento absoluto en GFLOPS entre FxC y FxT para hilos y procesos.

La explicación más plausible de este comportamiento se relaciona con la localidad de caché. En la multiplicación clásica, una de las matrices suele recorrerse con un patrón menos favorable para la memoria caché, lo que puede incrementar fallos de caché y accesos costosos a memoria principal. La transposición previa, en cambio, favorece accesos más secuenciales y coherentes con la jerarquía de memoria, lo que mejora el throughput absoluto.

La Figura 13 complementa esta interpretación al mostrar el comportamiento general del rendimiento por configuración. Allí se observa que no existe una única configuración dominante en todos los escenarios: el mejor resultado depende de la combinación entre tamaño, modelo de paralelismo, número de workers y variante algorítmica.

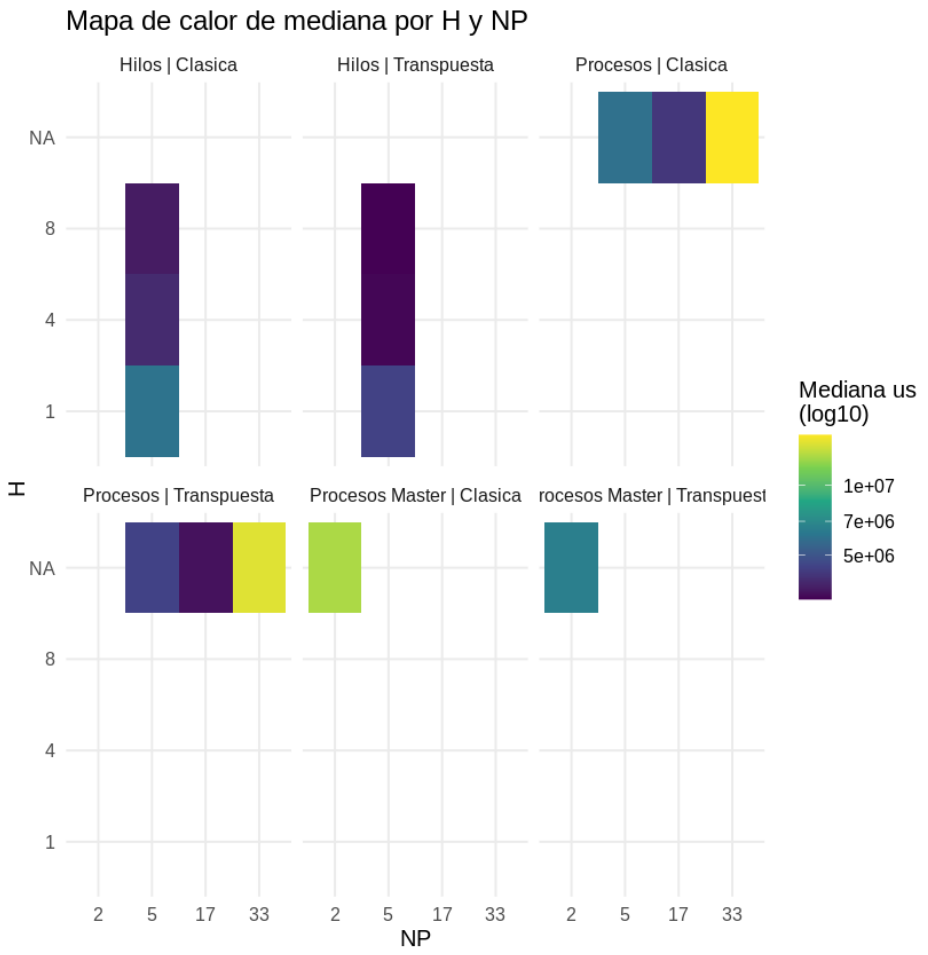

Figura 13: Mapa de calor del rendimiento por configuración experimental.

En consecuencia, la elección entre FxC y FxT depende del criterio de optimización. Si el objetivo es maximizar la ganancia relativa frente a una configuración base, FxC resulta más favorable. Si el objetivo es maximizar el rendimiento bruto en términos de GFLOPS, FxT ofrece mejores resultados. Desde el punto de vista de arquitectura de computadores, este resultado resalta la importancia de la organización del acceso a memoria, incluso en problemas donde la complejidad algorítmica asintótica no cambia.

6.6. Limitaciones del Experimento

Aunque los resultados obtenidos permiten identificar tendencias claras, el experimento presenta varias limitaciones que deben tenerse en cuenta al interpretar sus conclusiones.

En primer lugar, el espacio de configuración no es completamente simétrico entre hilos y procesos. No en todos los casos se evaluaron exactamente los mismos niveles de workers, lo que introduce cierta heterogeneidad en la comparación directa. Esta limitación no invalida el análisis, pero sí obliga a interpretar algunas diferencias con cautela.

En segundo lugar, existen variantes adicionales de procesos, incluyendo configuraciones asociadas a procesos master, que agregan complejidad al conjunto de datos. Esto amplía el espectro experimental, pero también puede introducir fuentes adicionales de variación que no están presentes en el modelo con hilos.

En tercer lugar, aunque el encabezado original de los datos sugería milisegundos, las mediciones fueron interpretadas como microsegundos según la validación experimental realizada. Esta aclaración es importante para evitar inconsistencias en la lectura de las magnitudes y en la comparación de métricas derivadas.

Asimismo, en algunas filas del modo procesos el valor H aparecía vacío, por lo que se asumió $H = 1$ para el cálculo del número de workers. Esta decisión es razonable desde el punto de vista del modelado, pero sigue siendo una suposición metodológica y, por tanto, una potencial fuente de sesgo menor.

Otra limitación relevante es la ausencia de instrumentación a nivel de sistema. No se midieron variables como afinidad de CPU, topología NUMA, tráfico de red, uso de caché, contención de memoria o interferencia con otros procesos del sistema. Por ello, el overhead debe interpretarse de manera inferencial y no como una descomposición exacta de costos internos.

Finalmente, los resultados representan el entorno específico de hardware y ejecución en el que se realizaron las pruebas. En consecuencia, su generalización a otros equipos, arquitecturas o topologías de memoria requiere validación adicional. Aun así, las tendencias observadas son consistentes con principios ampliamente conocidos en computación paralela: el costo de coordinación limita la escalabilidad, la granularidad del problema afecta el aprovechamiento del paralelismo y la localidad de memoria puede alterar significativamente el rendimiento absoluto.

6.7. Discusión General

Tomando en conjunto las métricas analizadas, se concluye que el modelo con hilos presenta el mejor balance global entre tiempo, estabilidad y escalabilidad. Aunque no siempre domina en cada métrica puntual, sí muestra un comportamiento más consistente a través de tamaños de problema y configuraciones de workers.

Por su parte, el modelo con procesos evidencia una mayor sensibilidad al overhead. Esto no implica que sea inherentemente inadecuado, sino que, en las condiciones de este experimento, los costos adicionales de coordinación reducen su competitividad frente a hilos. Es posible que en otros contextos, con diferente arquitectura, afinidad controlada o particionamientos más ajustados, su desempeño mejore.

Respecto a las variantes algorítmicas, la comparación entre FxC y FxT demuestra que la optimización del acceso a memoria puede ser tan determinante como la estrategia de paralelización. FxC obtiene mejores mejoras relativas frente a su baseline, pero FxT logra mayor throughput absoluto. Este resultado destaca que, al analizar desempeño, no basta con observar una sola métrica: speedup, eficiencia y GFLOPS ofrecen perspectivas complementarias y necesarias.

En suma, el experimento muestra que el rendimiento en multiplicación paralela de matrices depende de la interacción entre tamaño del problema, mecanismo de concurrencia, cantidad de workers y patrón de acceso a memoria. La mejor configuración no es universal, pero sí puede afirmarse que, dentro del espacio evaluado, las soluciones con hilos y una cuidadosa consideración de la localidad de memoria ofrecen la combinación más favorable de rendimiento y estabilidad.

7. Conclusiones y Recomendaciones

- **Conclusión y recomendación asociada al objetivo general.** La evaluación comparativa permitió establecer que el rendimiento de la multiplicación de matrices depende de la combinación entre el modelo de paralelismo, el tamaño del problema y el patrón de acceso a memoria. En el entorno experimental analizado, OpenMP y MPI ofrecieron el mejor balance global entre tiempo de ejecución, estabilidad y eficiencia.

- **Conclusión y recomendación asociada al objetivo específico 1: comparar dos algoritmos de multiplicación de matrices.** La comparación entre FxC y FxT mostró que no existe un algoritmo universalmente superior en todas las métricas: FxC alcanzó mejores mejoras relativas frente a su baseline, pero FxT obtuvo el mayor rendimiento absoluto en GFLOPS. Esto confirma que la organización del acceso a memoria influye de manera decisiva en el desempeño final.

- **Conclusión y recomendación asociada al objetivo específico 2: implementar y evaluar versiones secuenciales y paralelas con C, MPI y OpenMP.** Las implementaciones desarrolladas permitieron verificar que el paralelismo sí reduce el tiempo de ejecución cuando el tamaño de la matriz es suficientemente grande; sin embargo, el speedup real se mantuvo claramente por debajo del ideal. En particular, el mejor speedup observado fue de 2.39 en hilos y 1.65 en procesos, lo que evidencia que la ganancia efectiva está limitada por overheads de sincronización, distribución y comunicación.

- **Conclusión y recomendación asociada al objetivo específico 3: analizar el comportamiento en sistemas Linux.** La ejecución sobre un clúster homogéneo con Linux permitió observar un comportamiento consistente con la teoría; la homogeneidad del hardware ayudó a reducir sesgos experimentales.

- **Conclusión y recomendación asociada al objetivo específico 4: diseñar y ejecutar una batería de experimentos.** El diseño experimental con 56 archivos de datos y 1680 ejecuciones permitió caracterizar con suficiente solidez estadística el efecto del número de workers, del número de hilos y del tamaño de las matrices. Esta amplitud experimental hizo posible identificar zonas donde distribuir sí conviene, especialmente en tamaños grandes, y también configuraciones donde aumentar recursos ya no produce beneficios proporcionales.

- **Conclusión y recomendación asociada al objetivo específico 5: interpretar los resultados mediante métricas de desempeño.** El análisis conjunto de tiempo, speedup, eficiencia, overhead y GFLOPS mostró que una sola métrica no basta para describir el comportamiento del sistema. Aunque algunas configuraciones redujeron el tiempo total, su eficiencia fue baja y su overhead elevado; además, los resultados sugieren que la red y la comunicación entre procesos jugaron un papel importante en la degradación del rendimiento de MPI.

Referencias

[1] M. van Steen y A. S. Tanenbaum, "A brief introduction to distributed systems," *Computing*, vol. 98, no. 10, pp. 967–1009, 2016.

[2] G. Coulouris, J. Dollimore, T. Kindberg y G. Blair, "Distributed Systems: Concepts and Design," 5th ed., Addison-Wesley, 2011.

[3] A. Silberschatz, P. B. Galvin y G. Gagne, "Operating System Concepts," 10th ed., Wiley, 2018.

[4] B. Barney, "Introduction to parallel computing tutorial," Lawrence Livermore National Laboratory. [En línea]: https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial

[5] MPI Forum, "MPI: A Message-Passing Interface Standard, Version 4.1," nov. 2023. [En línea]: https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf

[6] Open MPI Project, "Open MPI: Open Source High Performance Computing." [En línea]: https://www.open-mpi.org/

[7] A. Grama, G. Karypis, V. Kumar y A. Gupta, *Introduction to Parallel Computing*, 2.ª ed. Boston, MA: Addison Wesley, 2003.

[8] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, 2021. [En línea]: https://www.openmp.org/specifications/